

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Федеральное государственное автономное образовательное учреждение
высшего образования**

**«Уральский федеральный университет имени первого Президента
России Б.Н. Ельцина»**

МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ

Направление подготовки: 09.04.01 «Информатика и вычислительная техника»

Тема: «Этика в искусственном интеллекте»

Студент:

Сорокин П.И.

Группа: РИМ-240910

Руководитель:

Аксенов К.А.

Екатеринбург 2026

СОДЕРЖАНИЕ

СПИСОК УСЛОВНЫХ ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ	5
АННОТАЦИЯ	7
ABSTRACT	9
ВВЕДЕНИЕ	10
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ ИССЛЕДОВАНИЯ	14
1.1 Проблема алгоритмического смещения в интеллектуальных системах принятия решений	14
1.2 Анализ нормативно-правового регулирования как системы граничных условий	16
1.3 Сравнительный анализ инструментальных средств аудита и коррекции моделей машинного обучения	18
1.4 Постановка задачи диссертационного исследования	20
Выводы по Главе 1	21
2 МАТЕМАТИЧЕСКИЕ МОДЕЛИ И МЕТОДЫ ОБЕСПЕЧЕНИЯ АЛГОРИТМИЧЕСКОЙ СПРАВЕДЛИВОСТИ	23
2.1 Формализация метрик групповой справедливости в задачах бинарной классификации	23
2.1.1 Критерий независимости (Independence)	24
2.1.2 Критерий разделимости (Separation)	24
2.1.3 Теорема о невозможности (Impossibility Theorem)	25
2.2 Теоретико-информационный подход к оценке зависимости (MINE)	26
2.2.1 Взаимная информация и дивергенция Кульбака-Лейблера	26
2.2.2 Проблема вычислительной сложности (Intractability)	26
2.2.3 Нижняя граница Донскера-Варадхана (DV-bound)	27
2.2.4 Нейросетевая аппроксимация (MINE)	27
2.3 Алгоритм состязательного дебиасинга	28

2.3.1	Постановка задачи минимаксной оптимизации	28
2.3.2	Итеративный алгоритм альтернативного градиентно- го спуска	29
2.3.3	Численная стабильность функции потерь	29
	Выводы по Главе 2	30
3	АРХИТЕКТУРНО-ТЕХНОЛОГИЧЕСКИЕ РЕШЕНИЯ И ПРО- ЕКТИРОВАНИЕ КОМПЛЕКСА	32
3.1	Проектирование архитектуры «FairnessSDK»: принципы мо- дульности и расширяемости	32
3.2	Алгоритмическое обеспечение ETL-процессов и инженерия признаков	37
3.3	Проектирование асинхронного API-шлюза	39
3.4	Подсистема информационной безопасности	41
	Выводы по Главе 3	42
4	ПРОГРАММНАЯ РЕАЛИЗАЦИЯ КОМПОНЕНТОВ КОМПЛЕКСА	44
4.1	Программная реализация ядра состязательного обучения . .	44
4.1.1	Класс Predictor (Генеративная сеть)	44
4.1.2	Класс MultiMINE (Сеть-Критик)	45
4.1.3	Класс MINEAdversarialTrainer	46
4.2	Реализация сервисного слоя и API-шлюза	48
4.3	Реализация модуля аудита и метрического контроля	52
4.4	Реализация механизмов информационной безопасности . . .	55
4.4.1	Контроль целостности (Integrity Control)	55
4.4.2	Безопасная десериализация (Secure Deserialization) . .	56
	Выводы по Главе 4	58
5	ЭКСПЕРИМЕНТАЛЬНЫЕ ИССЛЕДОВАНИЯ И АНАЛИЗ ЭФ- ФЕКТИВНОСТИ КОМПЛЕКСА	59
5.1	Методика проведения эксперимента и метрический базис . .	59
5.2	Анализ динамики обучения и сходимости алгоритма	62
5.3	Сравнительный анализ эффективности алгоритмов (A/B те- стирование)	63

5.4	Исследование чувствительности и выбор рабочей точки (Ablation Study)	67
5.5	Оценка производительности асинхронной архитектуры	68
	Выводы по Главе 5	68
6	ОБЕСПЕЧЕНИЕ ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ И ЭКСПЛУАТАЦИЯ КОМПЛЕКСА	70
6.1	Анализ векторов атак и построение модели угроз	70
6.1.1	Вектор AML.T0024: Компрометация цепочки поставок (Model Tampering)	70
6.1.2	Вектор AML.T0006: Удаленное исполнение кода через десериализацию (Establish Foothold via Deserialization)	71
6.2	Программная реализация контура защиты (Security Controls)	72
6.2.1	Защита от модификации моделей (Integrity Control)	72
6.2.2	Защита от атак десериализации (Secure Deserialization)	73
6.3	Регламент эксплуатации и ролевая модель доступа	75
	Выводы по Главе 6	76
	ЗАКЛЮЧЕНИЕ	77
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	80
А	Модуль состязательного обучения (fair_train_mine.py)	85
Б	Модуль обеспечения безопасности (security_manager.py)	91
В	Сервис API-шлюза (main.py)	93
Г	Модуль асинхронной обработки (tasks.py)	96
Д	Модуль расчета метрик (auditor.py)	99
Е	Контракты данных API (schemas.py)	102
Ж	Скрипт автоматизации эксперимента (run_tasks.py)	104
З	Скрипт тестирования контура безопасности	107

СПИСОК УСЛОВНЫХ ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ

Общие технические термины

- **ИИ** — Искусственный интеллект (*Artificial Intelligence*).
- **МО (ML)** — Машинное обучение (*Machine Learning*).
- **НС** — Нейронная сеть.
- **ПО** — Программное обеспечение.
- **БД** — База данных.
- **ОЗУ (RAM)** — Оперативное запоминающее устройство.
- **SDK** — *Software Development Kit* (Набор инструментов для разработки программного обеспечения).
- **API** — *Application Programming Interface* (Программный интерфейс приложения).

Специализированные термины (Fairness & Math)

- **MINE** — *Mutual Information Neural Estimation* (Нейросетевая оценка взаимной информации).
- **DI** — *Disparate Impact* (Несоразмерное воздействие — метрика алгоритмической справедливости).
- **SPD** — *Statistical Parity Difference* (Разность статистического паритета).
- **DV-bound** — *Donsker-Varadhan bound* (Нижняя граница Донскера-Варадхана для оценки дивергенции Кульбака-Лейблера).
- **TPR / FPR** — *True Positive Rate / False Positive Rate* (Доля истинно положительных / ложно положительных классификаций).

Архитектура и MLOps

- **REST** — *Representational State Transfer* (Архитектурный стиль взаимодействия компонентов).
- **ETL** — *Extract, Transform, Load* (Процесс извлечения, трансформации и загрузки данных).
- **RPS** — *Requests Per Second* (Количество запросов в секунду).
- **CI/CD** — *Continuous Integration / Continuous Deployment* (Непрерывная интеграция и доставка).
- **GPU** — *Graphics Processing Unit* (Графический процессор).

- **OOM** — *Out Of Memory* (Состояние нехватки оперативной памяти).
- **Информационная безопасность и Стандарты**
- **ИБ** — Информационная безопасность.
- **RBAC** — *Role-Based Access Control* (Управление доступом на основе ролей).
- **SHA-256** — *Secure Hash Algorithm 256-bit* (Алгоритм криптографического хеширования).
- **JWT** — *JSON Web Token* (Стандарт для создания токенов доступа).
- **RCE** — *Remote Code Execution* (Удаленное исполнение произвольного кода).
- **MITRE ATLAS** — *Adversarial Threat Landscape for AI Systems* (База знаний векторов атак на системы ИИ).
- **152-ФЗ** — Федеральный закон Российской Федерации «О персональных данных».
- **GDPR** — *General Data Protection Regulation* (Общий регламент по защите данных Европейского Союза).

АННОТАЦИЯ

Тема: Этика в искусственном интеллекте.

Объем работы: диссертация содержит 108 страниц, 4 рисунка, 2 таблицы, 45 источников литературы.

Объект исследования: процессы автоматизированного принятия решений в интеллектуальных информационных системах.

Предмет исследования: алгоритмы минимизации статистического смещения и программные средства обеспечения справедливости и информационной безопасности моделей машинного обучения.

Цель работы: разработка защищенного программного комплекса «FairnessSDK», обеспечивающего минимизацию дискриминации в алгоритмах классификации при сохранении предиктивной точности и выполнении требований к безопасности кода.

Методология и методы исследования. В работе применена математическая модель состязательного обучения (Adversarial Training), где в качестве регуляризатора функции потерь использована нейросетевая оценка взаимной информации (MINE — Mutual Information Neural Estimation). Данный подход позволяет минимизировать нелинейные зависимости между прогнозом модели и защищаемым атрибутом. Архитектурное проектирование выполнено на основе принципов микросервисной архитектуры и паттерна Producer-Consumer.

Результаты работы:

- Спроектирована и программно реализована микросервисная архитектура комплекса на стеке Python, FastAPI и Celery с использованием брокера сообщений Redis для асинхронной обработки задач аудита.
- Разработан алгоритм состязательного дебиасинга, обеспечивающий сходимость минимаксной игры между предиктором и критиком за счет применения взвешенной функции потерь и стабилизации градиентов (LogSumExp).
- Реализован контур информационной безопасности, включающий валидацию целостности ML-моделей (SHA-256), защиту от атак десериализации (Safe Unpickler) и ролевую модель доступа (RBAC).
- Экспериментально подтверждена эффективность разработанных мето-

дов на эталонном датасете UCI Adult: достигнуто повышение метрики справедливости (Disparate Impact) с дискриминационного уровня 0.617 до нормативного значения 0.932 при снижении точности классификации ($F1$ -score) менее чем на 8%.

Область применения: банковский скоринг, системы кадрового аудита, государственные сервисы автоматизированного принятия решений.

Ключевые слова: АЛГОРИТМИЧЕСКАЯ СПРАВЕДЛИВОСТЬ, МАШИННОЕ ОБУЧЕНИЕ, СОСТЯЗАТЕЛЬНОЕ ОБУЧЕНИЕ, MINE, MLOPS, ИНФОРМАЦИОННАЯ БЕЗОПАСНОСТЬ, МИКРОСЕРВИСЫ.

ABSTRACT

Topic: Ethics in Artificial Intelligence.

Volume: the thesis contains 108 pages, 4 figures, 2 tables, 45 references.

Object of study: automated decision-making processes in intelligent information systems.

Subject of study: algorithms for statistical bias minimization and software tools for ensuring fairness and security of machine learning models.

Objective: development of the secure software complex “FairnessSDK”, designed to minimize discrimination in classification algorithms while maintaining predictive accuracy and meeting code security requirements.

Methodology. The research utilizes an adversarial training mathematical model employing Mutual Information Neural Estimation (MINE) as a loss function regularizer. This approach enables the minimization of nonlinear dependencies between model predictions and protected attributes. The architectural design is based on microservice principles and the Producer-Consumer pattern.

Results:

- A microservice architecture was designed and implemented using the Python, FastAPI, and Celery stack, utilizing a Redis message broker for asynchronous audit task processing.
- An adversarial debiasing algorithm was developed, ensuring the convergence of the minimax game between the predictor and the critic networks through weighted loss function application and gradient stabilization (LogSumExp).
- A cybersecurity perimeter was implemented, including ML model integrity validation (SHA-256), protection against deserialization attacks (Safe Unpickler), and Role-Based Access Control (RBAC).
- The efficiency of the developed methods was empirically validated on the UCI Adult benchmark dataset: the fairness metric (Disparate Impact) was improved from a discriminatory level of 0.617 to a compliant value of 0.932, with a classification accuracy ($F1$ -score) decrease of less than 8%.

Applicability: credit scoring, HR audit systems, automated government decision-making services.

Keywords: ALGORITHMIC FAIRNESS, MACHINE LEARNING, ADVERSARIAL TRAINING, MINE, MLOPS, INFORMATION SECURITY, MICROSERVICES.

ВВЕДЕНИЕ

Актуальность темы исследования. В настоящее время интеллектуальные системы поддержки принятия решений (СППР) активно применяются в финансовом секторе, HR-индустрии и государственных сервисах. В основе таких систем лежат алгоритмы машинного обучения, способные обрабатывать большие объемы данных для задач классификации и скоринга.

Однако на практике использование стохастических моделей часто приводит к проблеме алгоритмического смещения (algorithmic bias). Модели начинают систематически занижать оценки для определенных групп пользователей, выделенных по признаку пола, возраста или этнической принадлежности. Это происходит потому, что обучающие выборки содержат исторические диспропорции. Нейронная сеть просто находит эти диспропорции в процессе минимизации ошибки и закрепляет их в своих весах. Как следствие, алгоритм начинает принимать дискриминационные решения.

Для решения данных проблем ужесточаются нормативно-правовые требования к системам ИИ. В частности, утвержден национальный стандарт ГОСТ Р 59276-2020 «Системы искусственного интеллекта. Способы обеспечения доверия». Он прямо требует обеспечить предсказуемость алгоритмов и минимизировать предвзятость.

Несмотря на наличие теоретических разработок, существующий инструментарий аудита и коррекции моделей (IBM AI Fairness 360, Microsoft Fairlearn) ориентирован преимущественно на исследовательские задачи (Data Science R&D) и не адаптирован к требованиям промышленной эксплуатации. Ключевыми недостатками существующих решений являются: отсутствие поддержки асинхронной обработки запросов, низкая производительность при работе с большими данными, использование линейных методов коррекции, не способных устранять сложные зависимости, и полное игнорирование аспектов информационной безопасности ML-моделей (защита от подмены и внедрения вредоносного кода).

В связи с этим, разработка защищенного программного комплекса, обеспечивающего автоматизированный аудит и нелинейную коррекцию смещения в контуре промышленной эксплуатации (Production), является актуальной научно-технической задачей, имеющей важное значение для развития

отечественных технологий доверенного искусственного интеллекта.

Объектом исследования выступают процессы автоматизированного принятия решений в интеллектуальных информационных системах, функционирующих на основе алгоритмов машинного обучения.

Предметом исследования являются математические методы минимизации статистического смещения, а также алгоритмы и программные средства обеспечения справедливости и информационной безопасности моделей искусственного интеллекта.

Цель работы заключается в разработке защищенного программного комплекса для аудита и коррекции алгоритмического смещения в моделях машинного обучения, обеспечивающего выполнение нормативных требований к доверенному искусственному интеллекту в условиях промышленной эксплуатации.

Для достижения поставленной цели решены следующие **задачи исследования**:

1. Провести системный анализ проблемы алгоритмического смещения и критический обзор существующих инструментальных средств, выявив их архитектурные и функциональные ограничения (Глава 1).
2. Разработать математическую модель состязательного обучения (Adversarial Training) с использованием нейросетевой оценки взаимной информации (MINE) в качестве регуляризатора функции потерь для минимизации нелинейных зависимостей (Глава 2).
3. Спроектировать микросервисную архитектуру программного комплекса и разработать алгоритмы предварительной обработки данных, обеспечивающие корректность обучения (Глава 3).
4. Выполнить программную реализацию компонентов системы, включая ядро состязательного обучения, асинхронный API-шлюз и модуль аудита метрик справедливости (Глава 4).
5. Провести экспериментальное исследование эффективности разработанного метода на эталонных наборах данных, подтвердив достижение нормативных показателей справедливости (Глава 5).
6. Разработать модель угроз и реализовать комплекс мер по обеспечению информационной безопасности и регламентации эксплуатации программного средства (Глава 6).

Научная новизна полученных результатов заключается в следующем:

1. Предложен метод регуляризации нейронных сетей в задачах бинарной классификации, отличающийся использованием оценки взаимной информации на основе нижней границы Донскера-Варадхана (DV-bound) в рамках минимаксной игры, что позволяет, в отличие от корреляционных методов, эффективно устранять нелинейные зависимости между прогнозом модели и защищаемым атрибутом.
2. Разработан алгоритм безопасной загрузки ML-моделей, отличающийся применением механизма фильтрации классов при десериализации (Allowlist Unpickling), что обеспечивает защиту от специфических векторов атак (RCE via Pickle) без потери производительности.

Теоретическая и практическая значимость работы. Теоретическая значимость заключается в развитии методов обучения представлений (Representation Learning), инвариантных к защищаемым атрибутам, и адаптации теоретико-информационных метрик для задач прикладного машинного обучения.

Практическая значимость работы состоит в создании готового к внедрению программного комплекса «FairnessSDK». Разработанное решение:

- Обеспечивает повышение метрики справедливости Disparate Impact до уровня 0.932 (при норме ≥ 0.8), что соответствует полному устранению дискриминации.
- Реализует микросервисную архитектуру, пригодную для развертывания в высоконагруженных средах.
- Содержит встроенные механизмы защиты от кибератак на цепочку поставок моделей (Integrity Control), что позволяет использовать комплекс в системах с повышенными требованиями к безопасности (банковский сектор, госуслуги).

Методология и методы исследования. Методологическую основу исследования составили методы системного анализа, теории вероятностей и математической статистики, теории информации (в части оценки энтропии и взаимной информации), методы оптимизации и машинного обучения (глубокие нейронные сети). При проектировании программного комплекса использовались принципы объектно-ориентированного программирования, паттерны проектирования распределенных систем и методы контейнеризации.

Положения, выносимые на защиту:

1. Математическая модель состязательного обучения с регуляризатором на основе MINE, обеспечивающая минимизацию статистической зависимости между целевым прогнозом и защищаемым атрибутом при сохранении информативности признакового пространства.
2. Микросервисная архитектура программного комплекса аудита и коррекции, реализующая асинхронный конвейер обработки задач и обеспечивающая линейную масштабируемость под нагрузкой.
3. Алгоритм обеспечения целостности и безопасности ML-моделей, реализующий защиту от внедрения вредоносного кода и подмены весов на этапах хранения и инференса.
4. Результаты экспериментальной валидации, подтверждающие эффективность предложенного метода дебиасинга (достижение целевого коридора справедливости $DI \in [0.8; 1.25]$).

Апробация результатов. Основные результаты работы докладывались и обсуждались на научно-технических семинарах кафедры Интеллектуальных информационных систем УрФУ.

Структура и объем работы. Диссертация состоит из введения, шести глав, заключения, списка использованных источников и приложений. Объем диссертации составляет 108 страниц машинописного текста, включает 4 рисунка и 2 таблицы.

ГЛАВА 1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ ИССЛЕДОВАНИЯ

1.1 Проблема алгоритмического смещения в интеллектуальных системах принятия решений

Переход от жестко запрограммированных алгоритмов к моделям машинного обучения выявил проблему алгоритмического смещения. Это явление заключается в том, что прогнозные оценки модели систематически отклоняются от истинного распределения вероятностей для определенных подгрупп пользователей.

В рамках данного исследования смещение рассматривается не как абстрактная этическая концепция, а как формализованная статистическая аномалия. Пусть $X \in \mathbb{R}^n$ — пространство признаков объекта, $Y \in \{0, 1\}$ — целевая переменная, а $S \in \{0, 1\}$ — защищаемый атрибут (например, пол), который делит выборку на привилегированную ($S = 1$) и непривилегированную ($S = 0$) группы.

В идеальной (справедливой) системе условные распределения вероятностей целевого события должны быть инвариантны относительно S . Однако на практике наблюдается нарушение статистического паритета:

$$P(\hat{Y} = 1 \mid S = 0) \neq P(\hat{Y} = 1 \mid S = 1) \quad (1.1)$$

где $\hat{Y} = f(X, S)$ — прогноз модели. Данное неравенство обусловлено рядом факторов, возникающих на различных этапах жизненного цикла данных (Data Lifecycle).

Генезис смещения: Исторический аспект (Historical Bias)

Первопричиной смещения является тот факт, что обучающая выборка $D_{train} = \{(x_i, s_i, y_i)\}_{i=1}^N$ представляет собой лишь аппроксимацию генеральной совокупности, сформированную в прошлом. Если исторические процессы принятия решений содержали системные диспропорции, то эмпирическое распределение $P_{emp}(Y, S)$ будет наследовать эти смещения.

Математически это выражается в том, что модель, минимизирующая функцию эмпирического риска $\mathcal{L}(\theta)$, обучается воспроизводить условную ве-

роятность $P(Y|X)$ из прошлого, тем самым закрепляя существующие корреляции. Таким образом, высокая точность модели (Accuracy) на историческом датасете может свидетельствовать о высокой степени воспроизводства исторической дискриминации.

Смещение репрезентативности (Representation Bias)

Данный тип смещения возникает вследствие неравномерной плотности выборки $p(x)$ в пространстве признаков $\mathcal{X}$. Если для определенной группы $S = 0$ объем выборки недостаточен ($N_{S=0} \ll N_{S=1}$), то дисперсия оценки параметров модели $\text{Var}(\hat{\theta})$ для данной области пространства будет существенно выше.

Согласно закону больших чисел, сходимость эмпирического риска к истинному гарантируется только при $N \rightarrow \infty$. В условиях ограниченной выборки для миноритарных групп модель демонстрирует низкую обобщающую способность, что приводит к росту ошибок I и II рода (False Positives / False Negatives) именно для сегмента $S = 0$.

Проблема прокси-переменных (Proxy Variables) и теорема о «Проклятии размерности»

Исключение защищаемого атрибута (подход «Fairness through Unawareness») на практике не решает проблему из-за скрытых корреляций с другими признаками.

В пространствах высокой размерности ($n \gg 1$) наблюдается эффект мультиколлинеарности: другие признаки (например, почтовый индекс, история покупок, лингвистические паттерны) могут иметь высокую взаимную информацию с S . Следовательно, модель способна восстановить скрытую переменную S через нелинейную комбинацию прокси-переменных X_{proxy} . Это явление усугубляется «проклятием размерности» (Curse of Dimensionality), когда в разреженном пространстве признаков модель находит сложные, но ложные корреляции, позволяющие косвенно дискриминировать группы.

Петли обратной связи (Feedback Loops)

Особую опасность представляет динамический аспект функционирования интеллектуальных систем. Внедрение модели в промышленную эксплуатацию (Production) изменяет состояние среды. Решения модели $\hat{Y}_t$ влияют на формирование выборки для дообучения модели на шаге $t + 1$.

Возникает замкнутый цикл: если модель с высокой вероятностью отка-

зывает группе $S = 0$ в кредитах, то в будущей выборке D_{t+1} будет отсутствовать информация о платежеспособности этой группы (“Censored Data”). Это приводит к росту энтропии $H(Y|S = 0)$ и дальнейшему усилению смещения модели.

Ниже приведена схема причинно-следственных связей, иллюстрирующая процесс распространения смещения (Рисунок 1.1).

Рисунок 1.1 — Причинно-следственная диаграмма возникновения алгоритмического смещения

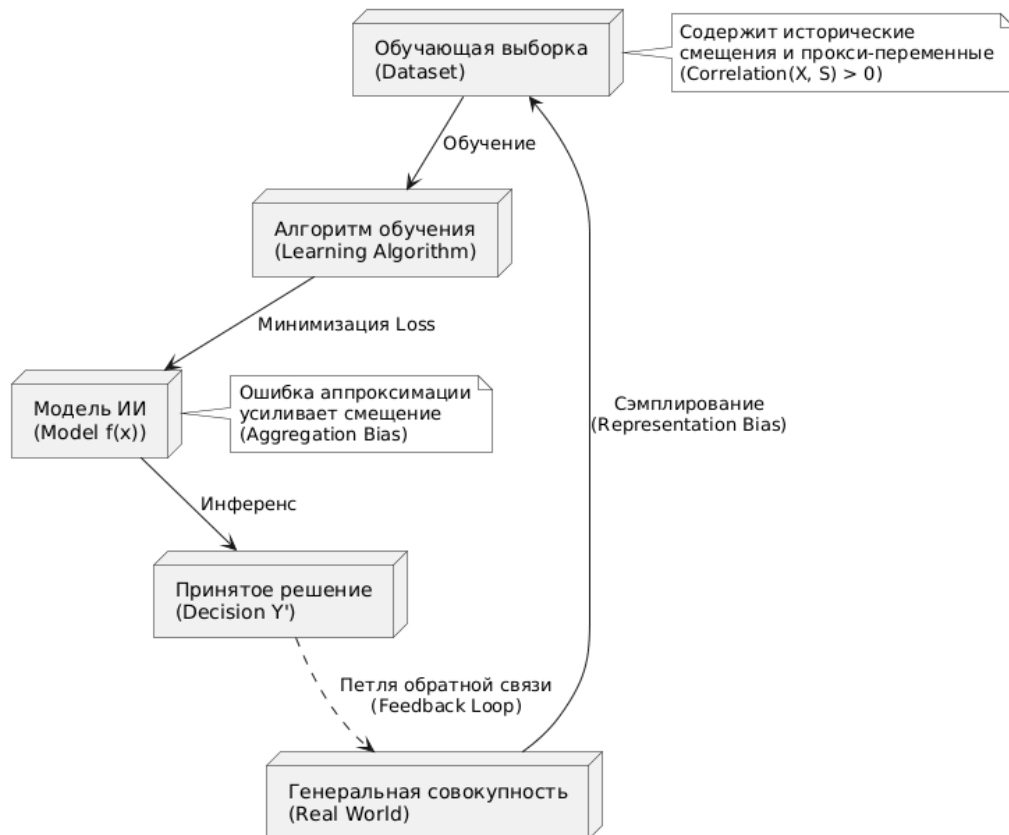


Рисунок 1.1 – Причинно-следственная диаграмма возникновения алгоритмического смещения

1.2 Анализ нормативно-правового регулирования как системы граничных условий

В современной парадигме разработки программного обеспечения нормативно-правовые акты перестают быть исключительно юридическими документами и трансформируются в набор жестких нефункциональных требований (Non-functional requirements, NFR) к архитектуре интеллектуальных систем. Проектирование системы “FairnessSDK” должно осуществляться в строгом соответствии с системой граничных условий, задаваемых междуна-

родными и национальными стандартами.

Международный уровень регулирования

Ключевым документом, определяющим требования к алгоритмам обработки данных, является General Data Protection Regulation (GDPR). В частности, Статья 22 («Automated individual decision-making, including profiling») закрепляет право субъекта не подвергаться решению, основанному исключительно на автоматизированной обработке. С инженерной точки зрения, это требование (Right to Explanation) накладывает ограничение на использование моделей типа «черный ящик» (Black Box) и требует интеграции модулей интерпретируемости (Explainable AI, XAI), способных предоставить локальную аппроксимацию функции принятия решения.

Кроме того, EU AI Act вводит классификацию систем ИИ на основе рисков (Risk-based approach). Системы скоринга и найма персонала отнесены к категории «High Risk», что требует обязательного технического аудита (Conformity Assessment), ведения журналов событий (Logging) и обеспечения надежности (Robustness) данных.

Национальная нормативная база РФ

В Российской Федерации стратегический вектор развития задан Указом Президента РФ от 10.10.2019 № 490 «О развитии искусственного интеллекта в Российской Федерации». Документ вводит понятие «Доверенного искусственного интеллекта», что подразумевает приоритет этических норм и безопасности над экономической эффективностью алгоритмов.

Для технической реализации данных принципов разработан комплекс стандартов, центральное место в котором занимает ГОСТ Р 59276-2020 «Системы искусственного интеллекта. Способы обеспечения доверия. Общие положения». Стандарт явно определяет характеристики доверия, которые должны быть метрически верифицируемы:

1. Предсказуемость: поведение системы должно оставаться стабильным при малых возмущениях входных данных ($|f(x + \delta) - f(x)| < \epsilon$).
2. Отсутствие дискриминации: требование минимизации предвзятости по защищаемым характеристикам.

Также необходимо учитывать требования Федерального закона № 152-ФЗ «О персональных данных». Согласно ст. 10, обработка специальных категорий персональных данных (раса, национальность, политические взгляды)

допускается только при наличии письменного согласия или в случаях, предусмотренных законом. Это накладывает ограничения на архитектуру хранения данных: защищаемые атрибуты S должны храниться в зашифрованном виде или в отдельном контуре безопасности, а доступ алгоритмов к ним должен быть строго регламентирован (Privacy by Design).

1.3 Сравнительный анализ инструментальных средств аудита и коррекции моделей машинного обучения

В настоящее время в индустрии разработки программного обеспечения (Software Engineering) и науке о данных (Data Science) сформировался класс инструментов, направленных на выявление и устранение алгоритмического смещения. К наиболее значимым решениям относятся библиотеки с открытым исходным кодом: IBM AI Fairness 360 (AIF360), Microsoft Fairlearn и инструмент визуализации Google What-If Tool.

Несмотря на широкую функциональность в части реализации базовых метрик справедливости, детальный архитектурный анализ выявляет их несоответствие требованиям промышленной эксплуатации (Production-ready) и интеграции в высоконагруженные MLOps-контуры.

Архитектурные ограничения и проблемы производительности

Большинство рассматриваемых аналогов (AIF360, Fairlearn) реализованы как синхронные Python-библиотеки, функционирующие в оперативной памяти (In-Memory). Обработка больших массивов данных (Big Data) осуществляется в блокирующем режиме, что неприемлемо для микросервисных архитектур, требующих асинхронной обработки запросов и горизонтального масштабирования. Отсутствие встроенных механизмов очередей задач (Task Queues) и распределенных вычислений приводит к тому, что интеграция данных инструментов в контур банка или телеком-оператора требует разработки сложной «обвязки» (wrapper), что увеличивает технический долг проекта.

Уязвимости информационной безопасности

Критическим недостатком существующих решений является полное игнорирование аспектов информационной безопасности ML-моделей (Adversarial Robustness).

1. Отсутствие контроля целостности: Библиотеки не предоставляют ме-

ханизмов криптографической верификации (хеширования) артефактов моделей. Это создает вектор атаки Model Hijacking, при котором злоумышленник может подменить веса нейросети на этапе развертывания.

2. Уязвимость десериализации: Стандартные механизмы загрузки (например, joblib или pickle в Scikit-learn/Fairlearn) подвержены атакам типа Insecure Deserialization (RCE), что недопустимо в системах класса Critical Infrastructure.

Математические ограничения методов коррекции

С точки зрения математического аппарата, большинство инструментов (в частности, Fairlearn) опираются на методы корреляционного анализа (Pearson Correlation) и линейные ограничения (Linear Constraints). Однако в глубоких нейронных сетях (Deep Neural Networks) зависимость между латентным представлением данных Z и защищаемым атрибутом S часто носит сложный нелинейный характер. Линейные методы дебиасинга не способны минимизировать взаимную информацию $I(Z; S)$ в случаях, когда $Cov(Z, S) \approx 0$, но Z и S статистически зависимы (например, квадратичная зависимость). Для решения этой проблемы требуется применение теоретико-информационных подходов, таких как MINE (Mutual Information Neural Estimation), которые отсутствуют в стандартных пакетах.

Сводный сравнительный анализ

Результаты сравнения функциональных возможностей существующих решений и разрабатываемого комплекса «FairnessSDK» представлены в Таблице 1.1.

Таблица 1.1 – Сравнительный анализ программных средств обеспечения доверенного ИИ

Критерий	IBM AIF360	Microsoft Fairlearn	Google What-If	FairnessSDK (Наша система)
Архитектура исполнения	Синхронная (Library)	Синхронная (Library)	Клиентская (JS/UI)	Асинхронная (Microservices + Celery)
Метод дебиасинга	Reweighting / Adversarial	Reduction (Linear)	Ручной подбор порогов	MINE (Non-linear Adversarial)
Работа с нелинейными зависимостями	Частично	Нет	Нет	Да (Mutual Info)
Контроль целостности (Integrity)	Нет	Нет	Нет	Да (SHA-256 + Ledger)
Защита от Pickle-атак	Нет	Нет	Нет	Да (SafeUnpickler)
Пригодность для High-Load	Низкая	Низкая	Низкая	Высокая

Вывод: На рынке отсутствует комплексное решение, объединяющее эффективный нелинейный дебиасинг и контур информационной безопасности. Это обосновывает необходимость разработки специализированного программного комплекса.

1.4 Постановка задачи диссертационного исследования

На основании проведенного анализа предметной области, выявленных проблем алгоритмического смещения и недостатков существующих инструментальных средств, формулируется цель и задачи исследования.

Цель работы: Разработка защищенного программного комплекса для аудита и коррекции алгоритмического смещения в моделях машинного обучения, обеспечивающего выполнение нормативных требований к доверенному искусственному интеллекту (ГОСТ Р 59276-2020) и информационную без-

опасность в условиях промышленной эксплуатации.

Для достижения поставленной цели необходимо решить следующие **задачи исследования**:

1. Разработать математическую модель состязательного обучения (Adversarial Training), использующую нейросетевую оценку взаимной информации (MINE — Mutual Information Neural Estimation) в качестве регуляризатора функции потерь для минимизации нелинейных зависимостей между прогнозом и защищаемым атрибутом.
2. Спроектировать и программно реализовать микросервисную архитектуру комплекса на базе фреймворка FastAPI и системы распределенных очередей Celery, обеспечивающую асинхронную обработку задач аудита и горизонтальную масштабируемость.
3. Реализовать программные модули обеспечения информационной безопасности, включая контроль целостности моделей (Integrity Control) на основе алгоритма SHA-256 и механизм безопасной десериализации (Safe Unpickler) для защиты от RCE-атак.
4. Провести экспериментальное исследование эффективности разработанного метода на эталонных наборах данных (UCI Adult Census), обеспечив достижение метрики Disparate Impact в диапазоне $DI \in [0.8; 1.25]$ при сохранении функциональной точности модели ($F1$ -score).

Выводы по Главе 1

В первой главе диссертационного исследования проведен системный анализ проблемы алгоритмического смещения в интеллектуальных системах. Показано, что смещение является следствием статистических аномалий в исторических данных и архитектурных особенностей нейронных сетей, способных восстанавливать скрытые зависимости.

Анализ нормативно-правовой базы (GDPR, ГОСТ Р 59276-2020) позволил сформулировать требования к системам ИИ как задачу условной оптимизации с ограничениями на справедливость и интерпретируемость. Критический обзор существующих инструментов выявил их непригодность для эксплуатации в защищенных средах, что подтверждает актуальность и научную новизну темы исследования.

Сформулированная цель и задачи определяют структуру дальнейшей

работы, направленной на создание архитектуры (Глава 2) и экспериментальную валидацию (Глава 3) системы «FairnessSDK».

ГЛАВА 2 МАТЕМАТИЧЕСКИЕ МОДЕЛИ И МЕТОДЫ ОБЕСПЕЧЕНИЯ АЛГОРИТМИЧЕСКОЙ СПРАВЕДЛИВОСТИ

В данной главе формулируется теоретико-вероятностная постановка задачи обеспечения справедливости (Fairness) в алгоритмах машинного обучения. Производится формализация метрик групповой справедливости, доказывается их несовместимость в общем случае (теоремы невозможности) и обосновывается выбор теоретико-информационного критерия на основе взаимной информации. Центральным элементом главы является математический вывод нижней границы Донскера-Варадхана, обеспечивающей возможность дифференцируемой оценки зависимости в нейросетевых архитектурах.

2.1 Формализация метрик групповой справедливости в задачах бинарной классификации

Пусть $(\Omega, \Sigma, \mathbb{P})$ — вероятностное пространство, описывающее генеральную совокупность данных. Рассмотрим следующие случайные величины, заданные на этом пространстве:

- $X : \Omega \rightarrow \mathbb{R}^n$ — вектор признаков объекта (предикторов);
- $S : \Omega \rightarrow \{0, 1\}$ — бинарный защищаемый атрибут (Sensitive Attribute), где $S = 1$ обозначает привилегированную группу, а $S = 0$ — непривилегированную;
- $Y : \Omega \rightarrow \{0, 1\}$ — истинная целевая переменная (Ground Truth);
- $\hat{Y} = f_{\theta}(X) : \Omega \rightarrow \{0, 1\}$ — прогноз классификатора, параметризуемого вектором θ .

Задача обеспечения алгоритмической справедливости сводится к наложению ограничений на совместное распределение вероятностей $(\hat{Y}, S, Y)$. В современной литературе выделяют два фундаментальных класса критериев: независимость и разделимость.

2.1.1 Критерий независимости (Independence)

Критерий независимости, также известный как Демографический паритет (Demographic Parity), требует, чтобы распределение прогнозов классификатора было статистически независимым от защищаемого атрибута:

$$\hat{Y} \perp S \quad (2.1)$$

В терминах условных вероятностей это условие записывается как равенство маргинальных вероятностей положительного исхода для обеих групп:

$$\mathbb{P}(\hat{Y} = 1 \mid S = 0) = \mathbb{P}(\hat{Y} = 1 \mid S = 1) \quad (2.2)$$

Для количественной оценки степени отклонения от данного условия вводятся следующие метрики:

1. Разность статистического паритета (Statistical Parity Difference, SPD):

$$SPD = \mathbb{E}[\hat{Y} \mid S = 0] - \mathbb{E}[\hat{Y} \mid S = 1] \quad (2.3)$$

Идеальным значением является $SPD = 0$. Отрицательное значение указывает на дискриминацию непривилегированной группы.

2. Коэффициент несоразмерного воздействия (Disparate Impact, DI):

$$DI = \frac{\mathbb{P}(\hat{Y} = 1 \mid S = 0)}{\mathbb{P}(\hat{Y} = 1 \mid S = 1)} \quad (2.4)$$

Метрика DI представляет собой отношение рисков. Значение $DI = 1$ соответствует полной независимости. В юридической практике США (рекомендации ЕЕОС) установлен диапазон допустимых значений $DI \in [0.8; 1.25]$.

2.1.2 Критерий разделимости (Separation)

Критерий разделимости, или Равные возможности (Equalized Odds), ослабляет требование полной независимости, допуская корреляцию прогноза с S , если она обусловлена истинной меткой Y . Формально требование записывается как условная независимость:

$$\hat{Y} \perp S \mid Y \quad (2.5)$$

Это условие эквивалентно одновременному равенству частот ошибок первого рода (False Positive Rate, FPR) и второго рода (в терминах True Positive Rate, TPR) для различных групп:

$$\forall y \in \{0, 1\} : \mathbb{P}(\hat{Y} = 1 \mid Y = y, S = 0) = \mathbb{P}(\hat{Y} = 1 \mid Y = y, S = 1) \quad (2.6)$$

Нарушение этого условия ведет к тому, что квалифицированные кандидаты ($Y = 1$) из разных групп имеют разные шансы на положительное решение алгоритма.

2.1.3 Теорема о невозможности (Impossibility Theorem)

Фундаментальным результатом, ограничивающим проектирование справедливых систем, является теорема Клейнберга-Чулдечовой (Kleinberg et al., 2016; Chouldechova, 2017).

Теорема. Пусть базовые уровни распространенности целевого признака (Base Rates) в группах различны, то есть $\mathbb{P}(Y = 1 \mid S = 0) \neq \mathbb{P}(Y = 1 \mid S = 1)$, и классификатор не является идеальным ($\hat{Y} \neq Y$). В таком случае невозможно одновременно удовлетворить критерии Независимости (Independence) и Разделимости (Separation).

Доказательство (схема): Если выполняется условие Разделимости ($\hat{Y} \perp S \mid Y$), то прогноз $\hat{Y}$ должен коррелировать с Y одинаково в обеих группах. Однако, поскольку Y коррелирует с S (по условию различия Base Rates), то через транзитивность корреляции $\hat{Y}$ также будет коррелировать с S , что нарушает условие Независимости ($\hat{Y} \perp S$).

Следствие: В условиях реальных данных (например, датасет Adult, где доход коррелирует с полом) необходимо выбирать *один* приоритетный критерий справедливости.

В рамках данного диссертационного исследования в качестве целевого критерия оптимизации выбран Disparate Impact (Independence). Это обусловлено тем, что минимизация зависимости $\hat{Y}$ от S позволяет устранить историческое смещение, заложенное в данных, даже ценой некоторого снижения точности аппроксимации Y (Accuracy-Fairness trade-off).

2.2 Теоретико-информационный подход к оценке зависимости (MINE)

Для реализации условия независимости $\hat{Y} \perp S$ в рамках градиентной оптимизации нейронных сетей необходимо ввести дифференцируемый функционал, оценивающий степень зависимости между латентным представлением данных $Z = E_\theta(X)$ и защищаемым атрибутом S . Классические меры (корреляция Пирсона) не способны улавливать нелинейные зависимости. В связи с этим применяется подход на основе Взаимной информации (Mutual Information).

2.2.1 Взаимная информация и дивергенция Кульбака-Лейблера

Взаимная информация $I(Z; S)$ определяется как мера уменьшения неопределенности (энтропии) одной случайной величины при получении информации о другой. Формально она выражается через дивергенцию Кульбака-Лейблера (D_{KL}) между совместным распределением $\mathbb{P}_{ZS}$ и произведением маргинальных распределений $\mathbb{P}_Z \otimes \mathbb{P}_S$:

$$I(Z; S) = D_{KL}(\mathbb{P}_{ZS} \parallel \mathbb{P}_Z \otimes \mathbb{P}_S) = \int_{Z \times S} \log \left(\frac{d\mathbb{P}_{ZS}}{d(\mathbb{P}_Z \otimes \mathbb{P}_S)} \right) d\mathbb{P}_{ZS} \quad (2.7)$$

Свойство $I(Z; S) \geq 0$, причем равенство достигается тогда и только тогда, когда Z и S статистически независимы.

2.2.2 Проблема вычислительной сложности (Intractability)

Прямое вычисление $I(Z; S)$ в задачах глубокого обучения сталкивается с проблемой высокой размерности пространства $\mathcal{Z}$. Интегрирование плотностей вероятностей $p(z, s)$ и $p(z)$ требует знания аналитического вида этих распределений, которые для скрытых слоев нейронной сети неизвестны. Непараметрические методы (binning, k-NN) не являются дифференцируемыми и неэффективны при большой размерности выборки («Проклятие размерности»).

2.2.3 Нижняя граница Донскера-Варадхана (DV-bound)

Для решения проблемы вычислимости используется вариационный подход. Основой метода MINE является двойственное представление дивергенции D_{KL} , известное как вариационная формула Донскера-Варадхана:

$$D_{KL}(\mathbb{P} \parallel \mathbb{Q}) = \sup_{T: \Omega \rightarrow \mathbb{R}} (\mathbb{E}_{\mathbb{P}}[T] - \log(\mathbb{E}_{\mathbb{Q}}[e^T])) \quad (2.8)$$

где супремум берется по классу всех измеримых функций T , для которых соответствующие математические ожидания конечны.

Применительно к взаимной информации, полагая $\mathbb{P} = \mathbb{P}_{ZS}$ и $\mathbb{Q} = \mathbb{P}_Z \otimes \mathbb{P}_S$, получаем нижнюю границу:

$$I(Z; S) \geq \sup_T \left(\mathbb{E}_{\mathbb{P}_{ZS}}[T(z, s)] - \log(\mathbb{E}_{\mathbb{P}_Z \otimes \mathbb{P}_S}[e^{T(z, s)}]) \right) \quad (2.9)$$

Эта формула позволяет перейти от задачи интегрирования к задаче оптимизации. Функция T выступает в роли «критика», который пытается различить совместное распределение (реальные пары (z, s)) и произведение маргинальных (где s не зависит от z).

2.2.4 Нейросетевая аппроксимация (MINE)

Метод Mutual Information Neural Estimation (MINE) (Belghazi et al., 2018) предлагает аппроксимировать пространство функций T параметрическим семейством нейронных сетей T_{ω} с параметрами $\omega \in \Omega$.

Тогда оценка взаимной информации сводится к максимизации по параметрам ω :

$$\hat{I}_{MINE}(Z; S) = \sup_{\omega} \left(\frac{1}{n} \sum_{i=1}^n T_{\omega}(z_i, s_i) - \log \left(\frac{1}{n} \sum_{i=1}^n e^{T_{\omega}(z_i, \bar{s}_i)} \right) \right) \quad (2.10)$$

где (z_i, s_i) — сэмплы из совместного распределения, а $\bar{s}_i$ — сэмплы из маргинального распределения (полученные путем случайной перестановки S внутри батча).

Полученная оценка $\hat{I}_{MINE}$ является дифференцируемой (через алгоритм обратного распространения ошибки) и состоятельной, что позволяет использовать её в качестве регуляризационного члена в функции потерь ос-

новой модели.

2.3 Алгоритм состязательного дебиасинга с MINE-регуляризацией

Сформулируем задачу обучения справедливой модели как задачу условной оптимизации, где целевая функция потерь $\mathcal{L}_{task}$ минимизируется при наложении ограничения на взаимную информацию $I(Z; S) \leq \epsilon$. В терминах метода множителей Лагранжа данная задача сводится к поиску седловой точки функционала, объединяющего ошибку классификации и оценку зависимости.

2.3.1 Постановка задачи минимаксной оптимизации

Рассмотрим процесс обучения как антагонистическую игру (Zero-Sum Game в части регуляризатора) между двумя параметрическими моделями:

1. **Предиктор (Predictor/Encoder)** $f_\theta : \mathcal{X} \rightarrow \mathcal{Z} \rightarrow \mathcal{Y}$. Параметризуется вектором $\theta \in \Theta$. Его цель — минимизировать эмпирический риск классификации $\mathcal{L}_{task}$ и одновременно минимизировать оценку взаимной информации $\hat{I}_{MINE}$, генерируя инвариантное латентное представление Z .
2. **Критик (Statistics Network)** $T_\omega : \mathcal{Z} \times \mathcal{S} \rightarrow \mathbb{R}$. Параметризуется вектором $\omega \in \Omega$. Его цель — максимизировать нижнюю границу Донскера-Варадхана, находя наиболее сильную статистическую зависимость между Z и S .

Целевой функционал (Lagrangian) системы определяется следующим образом:

$$V(\theta, \omega) = \mathcal{L}_{task}(\theta) + \lambda \cdot \hat{I}_{MINE}(\theta, \omega) \quad (2.11)$$

где $\lambda > 0$ — гиперпараметр регуляризации, определяющий баланс между точностью и справедливостью (Trade-off parameter).

Задача обучения сводится к поиску решения минимаксной задачи:

$$\theta^*, \omega^* = \arg \min_{\theta} \max_{\omega} V(\theta, \omega) \quad (2.12)$$

2.3.2 Итеративный алгоритм альтернативного градиентного спуска

Прямое решение задачи $\min \max$ аналитическими методами невозможно ввиду невыпуклости функций, реализуемых нейронными сетями. Для нахождения локального равновесия Нэша применяется итеративная процедура стохастического градиентного спуска (SGD) с чередованием обновлений (Alternating Gradient Descent).

Критически важным условием сходимости MINE является точность оценки взаимной информации на каждом шаге обновления предиктора. Если оценка $\hat{I}_{MINE}$ далека от истинного значения $I(Z; S)$, градиент $\nabla_{\theta} \hat{I}_{MINE}$ будет указывать ложное направление дебиасинга. В связи с этим, алгоритм реализует асимметричную схему обучения: на один шаг обновления параметров предиктора θ приходится $k > 1$ шагов обновления параметров критика ω .

Алгоритм 2.1 (Состязательное обучение с MINE):

1. Внутренний цикл (Обучение Критика): Фиксируем параметры θ . Выполняем k шагов градиентного подъема (Gradient Ascent) для ω :

$$\omega^{(t+1)} \leftarrow \omega^{(t)} + \eta_{\omega} \nabla_{\omega} \hat{I}_{MINE}(E_{\theta}(X); S) \quad (2.13)$$

Здесь максимизируется нижняя граница, что позволяет критику аппроксимировать истинную дивергенцию D_{KL} . В программной реализации выбрано $k = 5$.

2. Внешний цикл (Обучение Предиктора): Фиксируем параметры ω . Выполняем 1 шаг градиентного спуска (Gradient Descent) для θ :

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \eta_{\theta} \nabla_{\theta} \left(\mathcal{L}_{task}(\theta) + \lambda \cdot \hat{I}_{MINE}(\theta, \omega^{(t+1)}) \right) \quad (2.14)$$

На этом шаге предиктор обновляет веса так, чтобы «обмануть» оптимального критика, снижая оценку взаимной информации.

2.3.3 Обоснование численной стабильности функции потерь

При реализации градиентных методов в архитектурах глубокого обучения возникает проблема численной неустойчивости (Numerical Instability), связанная с переполнением (Overflow) или исчезновением (Underflow) значе-

ний экспоненциальной функции.

Для задачи бинарной классификации стандартная функция потерь $\mathcal{L}_{task}$ (Binary Cross-Entropy) определяется как:

$$BCE(p, y) = -(y \cdot \log(p) + (1 - y) \cdot \log(1 - p)), \quad \text{где } p = \sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.15)$$

Прямое вычисление $\log(\sigma(x))$ при больших значениях $|x|$ приводит к потере точности. В разработанном алгоритме применяется комбинированная функция `BCEWithLogitsLoss`, использующая прием **LogSumExp trick**. Рассмотрим преобразование для слагаемого $\log(1 - p)$:

$$\log(1 - \sigma(x)) = \log\left(1 - \frac{1}{1 + e^{-x}}\right) = \log\left(\frac{e^{-x}}{1 + e^{-x}}\right) = -x - \log(1 + e^{-x}) \quad (2.16)$$

Функция $\log(1 + e^{-x})$ программно аппроксимируется как $\max(0, -x) + \log(1 + e^{-|x|})$, что гарантирует отсутствие переполнения. Использование логитов (Logits) вместо вероятностей в качестве выхода сети f_θ обеспечивает стабильность градиентов $\nabla_\theta \mathcal{L}_{task}$ во всем диапазоне определений, что является необходимым условием сходимости состязательной игры.

Выводы по Главе 2

В данной главе разработано теоретико-методологическое обеспечение диссертационного исследования, формирующее математический фундамент системы «FairnessSDK»:

1. **Формализация метрик:** Даны строгие вероятностные определения критериев групповой справедливости. На основе теоремы Клейнберга о невозможности одновременного удовлетворения условий независимости и разделимости обоснован выбор метрики Disparate Impact (DI) в качестве целевого функционала оптимизации.
2. **Теоретико-информационный базис:** Обосновано применение взаимной информации $I(Z; S)$ как универсальной меры зависимости нелинейных случайных величин. Доказана корректность перехода от невычислимого интеграла Кульбака-Лейблера к вариационной нижней границе Донскера-Варадхана (DV-bound), аппроксимируемой нейронной сетью.

3. **Алгоритм оптимизации:** Разработана математическая модель состязательного обучения в виде минимаксной задачи. Сформулирован итеративный алгоритм с асимметричным шагом обновления (k -step update) для обеспечения сходимости оценки MINE.
4. **Численная реализация:** Обоснован выбор функции потерь на основе LogSumExp-преобразования, обеспечивающего численную устойчивость градиентов в условиях стохастической оптимизации.

Представленные математические модели и алгоритмы являются основой для проектирования архитектуры программного комплекса, описание которого приводится в следующей главе.

ГЛАВА 3

АРХИТЕКТУРНО-ТЕХНОЛОГИЧЕСКИЕ РЕШЕНИЯ И ПРОЕКТИРОВАНИЕ КОМПЛЕКСА

Третья глава диссертационного исследования посвящена системному анализу и проектированию программно-аппаратной архитектуры комплекса «FairnessSDK». В главе детально рассматриваются принципы построения распределенных вычислительных систем для задач машинного обучения, обосновывается выбор технологического стека через сравнительный анализ альтернатив, а также формализуются алгоритмы предварительной обработки данных (ETL), обеспечивающие математическую корректность работы методов дебиасинга.

3.1 Проектирование архитектуры «FairnessSDK»: принципы модульности и расширяемости

Проектирование высоконагруженных систем интеллектуального анализа данных требует отказа от монолитных архитектурных паттернов в пользу слабосвязанных распределенных компонентов. Архитектура комплекса «FairnessSDK» базируется на принципах слоистой архитектуры (Layered Architecture) и инверсии зависимостей (Dependency Inversion Principle, DIP), что обеспечивает изоляцию бизнес-логики от инфраструктурных деталей и возможность независимого масштабирования модулей.

Декомпозиция системы и анализ слоев

Система структурирована в виде четырех ортогональных слоев, взаимодействие между которыми строго регламентировано контрактами данных:

1. Слой интерфейсов (Interface Layer):

Отвечает за взаимодействие с внешними потребителями (клиентскими приложениями, CI/CD пайплайнами). Реализован в виде RESTful API Gateway. Основная задача слоя — трансляция HTTP-запросов во внутренние команды системы, валидация входных данных (Input Validation) и аутентификация. Этот слой не содержит бизнес-логики, что позволя-

ет легко заменять транспортные протоколы (например, перейти с REST на gRPC) без модификации ядра.

2. Слой приложения (Application Layer):

Выступает в роли оркестратора бизнес-процессов. Здесь сосредоточена логика управления жизненным циклом задач (Task Lifecycle Management): постановка экспериментов в очередь, мониторинг статусов, агрегация результатов аудита. Слой приложения координирует работу доменных сервисов, не вникая в детали их реализации.

3. Доменный слой (Domain Layer):

Ядро системы, инкапсулирующее математические модели и алгоритмы. Включает реализацию нейронных сетей (Predictor, MultiMINE), функции потерь (Loss Functions) и логику состязательного обучения. Согласно принципу DIP, этот слой не зависит от фреймворков веб-разработки или баз данных; он оперирует исключительно абстракциями тензорных вычислений.

4. Инфраструктурный слой (Infrastructure Layer):

Обеспечивает техническую реализацию механизмов персистентности и обмена сообщениями. Включает адаптеры к файловой системе (хранение весов моделей .pth), брокерам сообщений (Redis) и базам данных.

Асинхронная модель взаимодействия и выбор стека

Специфика задач машинного обучения (Machine Learning) заключается в наличии длительных блокирующих операций (CPU/GPU-bound), время выполнения которых несопоставимо с тайм-аутами веб-соединений. Обучение модели на большом датасете может занимать часы. В условиях однопоточной модели исполнения Python (ограниченной Global Interpreter Lock, GIL), синхронная обработка такого запроса привела бы к полному зависанию сервера (Thread Starvation) и отказу в обслуживании для остальных клиентов.

Для решения этой проблемы внедрена асинхронная модель на базе очередей сообщений.

Выбор веб-фреймворка: FastAPI vs Flask

В качестве основы API Gateway выбран фреймворк FastAPI.

- *Аргументация:* FastAPI построен на базе стандарта ASGI и использует событийный цикл (Event Loop) библиотеки uvicorn. Это позволяет обрабатывать тысячи конкурентных I/O-операций (запросы к БД, отправка

задач в очередь) в одном потоке, используя конструкцию `async/await`.

- *Сравнение:* Классический фреймворк Flask работает на синхронном WSGI-сервере. Для обработки параллельных запросов он требует создания отдельных потоков или процессов (Thread/Process Forking), что создает значительные накладные расходы на переключение контекста (Context Switching) и ограничивает максимальную пропускную способность (Throughput) ресурсами ОС.

Выбор брокера сообщений: Redis vs RabbitMQ

Для организации очереди задач используется Redis.

- *Аргументация:* Redis является in-memory хранилищем структур данных, обеспечивающим субмиллисекундную латентность (Low Latency). Для задач ML-оркестрации, где важна скорость передачи метаданных задачи, Redis является оптимальным выбором. Атомарные операции со списками (LPUSH, RPOP) гарантируют целостность очереди без сложных механизмов блокировок. Механизмы персистентности (RDB/AOF) обеспечивают сохранность задач при перезагрузке.
- *Сравнение:* RabbitMQ — мощный брокер с поддержкой сложных паттернов маршрутизации (AMQP), однако его настройка и эксплуатация значительно сложнее (Higher Operational Complexity). Для сценария Simple Task Queue функциональность RabbitMQ избыточна, а накладные расходы на протокол выше.

Оркестрация задач: Celery

В качестве менеджера распределенных задач выбрана библиотека Celery. Она реализует паттерн Producer-Consumer, позволяя полностью развязать процесс приема запроса (Producer) и его выполнения (Consumer). Это исключает риск тайм-аутов на клиенте и позволяет сглаживать пиковые нагрузки (Traffic Spikes), накапливая задачи в очереди.

Для наглядной демонстрации взаимодействия компонентов системы разработана схема развертывания (Deployment Diagram), представленная на Рисунке 3.1.

Схема иллюстрирует физическое разделение процессов и потоки данных в системе:

1. Клиент (Client): Иницирует запрос через REST API.
2. Server Host: Контейнеризированная среда, включающая:

- API Gateway: Принимает запрос и делегирует задачу.
 - Message Broker (Redis): Буферизирует задачи, обеспечивая надежность доставки.
 - ML Worker: Асинхронно извлекает задачу, запускает цикл обучения (MINE Loop) и сохраняет результат.
3. File System: Служит персистентным хранилищем для артефактов обученных моделей (.pth).

Такая архитектура гарантирует, что тяжелые вычисления ML-модели изолированы от контура обработки пользовательских запросов, что обеспечивает высокую доступность сервиса.

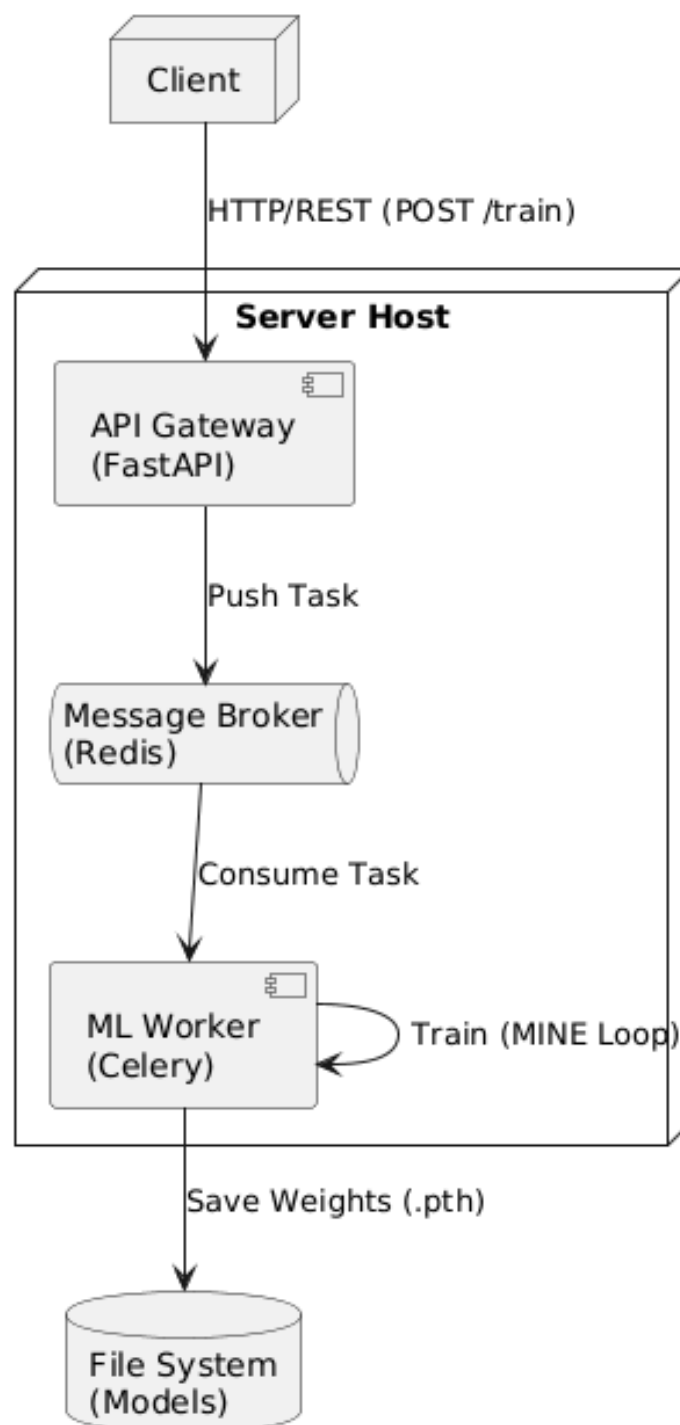


Рисунок 3.1 – Схема развертывания программного комплекса FairnessSDK

Масштабируемость (Scalability)

Архитектура воркеров (Celery Workers) спроектирована как Stateless (без сохранения состояния). Воркер получает весь необходимый контекст (пути к данным, гиперпараметры) из сообщения задачи. Это свойство является фундаментом горизонтальной масштабируемости. В среде оркестрации контейнеров (Kubernetes) увеличение производительности системы достига-

ется простым увеличением количества реплик (Pods) с воркерами. Система способна линейно наращивать вычислительную мощность при добавлении аппаратных ресурсов, что критически важно для Enterprise-решений.

3.2 Алгоритмическое обеспечение ETL-процессов и инженерия признаков

Эффективность алгоритмов машинного обучения, и в особенности методов дебиасинга, критически зависит от качества входных данных. В системе реализован строгий ETL-пайплайн (Extract, Transform, Load), обеспечивающий математическую корректность тензорных операций.

Анализ и очистка данных (Data Cleaning)

В качестве эталонного набора данных используется UCI Adult Census Income. Специфика датасета заключается в наличии пропущенных значений (Missing Values), маркированных символом ‘?’. В рамках предобработки принята стратегия удаления записей с пропусками (Listwise Deletion).

- *Обоснование:* Альтернативные методы, такие как импутация средним (Mean Imputation) или модой, неизбежно вносят статистическое искажение (Imputation Bias). Заполнение пропусков синтетическими значениями может искусственно усилить или ослабить корреляцию между признаками и целевой переменной. В задачах алгоритмической справедливости сохранение истинной структуры распределения данных (True Distribution Structure) является приоритетом над сохранением объема выборки (при условии, что доля пропусков невелика).

Кодирование признаков (Feature Encoding)

Преобразование категориальных переменных в численный вид требует дифференцированного подхода.

1. Защищаемый атрибут и Целевая переменная (Label Encoding):

Для полей sex (защищаемый атрибут S) и income (целевая переменная Y) используется Label Encoding (отображение в $\{0, 1\}$).

- *Обоснование:* В архитектуре MINE защищаемый атрибут подается на вход дискриминатора и используется для расчета условных вероятностей. Использование One-Hot Encoding (OHE) для бинарного признака привело бы к созданию вектора размерности 2 ($[1, 0]$

и $[0, 1]$), что избыточно и усложняет тензорные операции маскирования. Формат скаляра 0/1 позволяет использовать S как индекс или маску напрямую.

2. Остальные категориальные признаки (One-Hot Encoding):

Для признаков с низкой кардинальностью (например, `workclass`, `marital-status`) применяется ОНЕ. Это позволяет модели улавливать нелинейные зависимости без введения ложного порядка, характерного для Label Encoding (где $Category_A < Category_B$).

Масштабирование признаков (Feature Scaling)

Для численных признаков (`age`, `fnlwgt`, `capital-gain`, `hours-per-week`) применяется стандартизация (Z-score normalization):

$$x'_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j} \quad (3.1)$$

где μ_j и σ_j — выборочное среднее и стандартное отклонение j -го признака.

Математическое обоснование:

Градиентные методы оптимизации первого порядка (SGD, Adam) чувствительны к масштабу входных данных. Поверхность функции потерь (Loss Landscape) для признаков с разным масштабом (например, возраст $\in [17, 90]$ и доход $\in [0, 100000]$) приобретает форму вытянутого эллипсоида (“оврага”). Это приводит к тому, что градиент по одному направлению значительно больше, чем по другому, вызывая осцилляции и требуя крайне малого шага обучения. Стандартизация приводит линии уровня функции потерь к виду, близкому к сферическому, что обеспечивает быструю и стабильную сходимость оптимизатора.

Балансировка и стратификация

Датасет Adult является несбалансированным (Imbalanced): класс `>50K` составляет лишь $\approx 24\%$ выборки. При разделении данных на обучающую и тестовую выборки (Train/Test Split) критически важно использовать стратификацию (Stratified Sampling) по целевой переменной Y .

- *Обоснование:* Случайная выборка (Random Sampling) не гарантирует сохранения пропорций классов. Нарушение распределения в тестовой выборке сделает оценку метрик (особенно F1-score и Disparate Impact) статистически несостоятельной. Стратификация фиксирует маргиналь-

ное распределение $P(Y)$ во всех подвыборках, обеспечивая репрезентативность эксперимента.

3.3 Проектирование асинхронного API-шлюза и контрактов данных

Центральным компонентом взаимодействия в распределенной системе FairnessSDK является API-шлюз, обеспечивающий трансляцию внешних HTTP-запросов во внутренние команды оркестрации. Проектирование данного слоя выполнено с учетом требований к отказоустойчивости, высокой доступности (High Availability) и строгой типизации данных.

Паттерны асинхронного взаимодействия

Учитывая специфику задач глубокого обучения (Deep Learning), время выполнения которых варьируется от минут до нескольких часов, использование синхронной модели взаимодействия (Request-Response) является архитектурной ошибкой. Удержание открытого TCP-соединения (Keep-Alive) в течение длительного времени приводит к исчерпанию пула соединений на балансировщике нагрузки и высокой вероятности разрыва связи по тайм-ауту (Gateway Timeout).

В системе реализован гибридный паттерн взаимодействия:

1. Fire-and-Forget (Инициализация):

Метод POST /train реализует стратегию «отправил и забыл» с подтверждением приема. При получении запроса сервер не блокирует поток обработки. Вместо этого он выполняет сериализацию параметров задачи, помещает её в очередь брокера Redis и немедленно возвращает клиенту HTTP-ответ 202 Accepted с уникальным идентификатором `task_id`. Это гарантирует мгновенный отклик API (Latency < 50ms) независимо от сложности предстоящих вычислений.

2. Polling (Мониторинг):

Для получения результатов используется паттерн периодического опроса. Клиент обращается к эндпоинту GET /status/{task_id}.

- *Обоснование отказа от WebSocket:* Использование постоянных дуплексных соединений (WebSocket) потребовало бы усложнения инфраструктуры (Stateful-балансировка, управление разры-

вами соединений). Для задач пакетной обработки данных (Batch Processing), где статус меняется относительно редко (раз в эпоху), механизм Polling является достаточным и более надежным (Robust) архитектурным решением.

Контракты данных (DTO) и Валидация

Обеспечение целостности данных на границе системы реализовано через механизм Data Transfer Objects (DTO) с использованием библиотеки Pydantic. В отличие от стандартных словарей Python (dict), Pydantic выполняет парсинг и валидацию входного JSON-потока в момент инициализации объекта. Это позволяет гарантировать соответствие типов данных заявленному контракту.

Защита от Type Confusion и инъекций:

Строгая типизация полей (например, `learning_rate: float`) действует как первый эшелон защиты. Если злоумышленник попытается передать в числовом поле строку с SQL-инъекцией или скриптом, валидатор автоматически сгенерирует ошибку 422 Unprocessable Entity. Это превентивно блокирует целый класс атак, связанных с манипуляцией типами данных (Type Confusion), и гарантирует, что внутренние компоненты системы оперируют только корректными значениями.

Жизненный цикл задачи (Task Lifecycle)

Управление состоянием распределенных вычислений реализовано через конечный автомат (State Machine), состояние которого хранится в in-memory базе Redis. Диаграмма переходов состояний:

- **PENDING:** Задача принята API и ожидает в очереди (LPUSH).
- **STARTED:** Воркер извлек задачу (RPOP) и начал инициализацию контекста PyTorch. На этом этапе в Redis записывается PID процесса.
- **PROGRESS:** Промежуточное состояние, обновляемое каждую эпоху обучения. Содержит текущие метрики (Loss, F1).
- **SUCCESS:** Успешное завершение. В Redis сохраняется JSON-сериализованный отчет и путь к артефактам.
- **FAILURE:** Аварийное завершение (например, OOM - Out Of Memory). Сохраняется стек вызовов (Traceback) для отладки.

Использование Redis с настроенным временем жизни ключей (TTL - Time To Live) обеспечивает автоматическую «сборку мусора» и предотвращает утечки

памяти при накоплении метаданных старых задач.

3.4 Проектирование подсистемы информационной безопасности

Подсистема безопасности FairnessSDK спроектирована в парадигме Defense in Depth (Эшелонированная защита), с учетом специфических векторов атак на системы машинного обучения, описанных в матрице угроз MITRE ATLAS.

Модель угроз: Атаки на цепочку поставок ML

Ключевым вектором атаки на системы ИИ является компрометация артефактов (Supply Chain Attacks).

1. Model Tampering (AML.T0024): Злоумышленник, получивший доступ к файловому хранилищу, может модифицировать веса обученной модели. Изменение даже незначительного количества параметров может привести к внедрению «бэкдора» (Backdoor) или инверсии логики принятия решений, при этом структура файла останется валидной.
2. Arbitrary Code Execution (внедрение эксплойта): Формат сериализации pickle, являющийся стандартом де-факто в PyTorch, позволяет сохранять не только данные, но и инструкции по конструированию объектов. Это дает возможность внедрить в файл модели вредоносный байт-код, который будет исполнен интерпретатором при загрузке (`torch.load`).

Механизмы защиты (Security Controls)

1. Криптографический контроль целостности (Cryptographic Integrity)

Для защиты от модификации моделей внедрен механизм обязательной верификации хеш-сумм.

- Алгоритм: При сохранении модели вычисляется её дайджест по алгоритму SHA-256. Выбор SHA-256 обусловлен его криптографической стойкостью к коллизиям и высокой производительностью на современных процессорах с поддержкой инструкций Intel SHA-NI.
- Процедура: Перед загрузкой весов в память GPU система повторно вычисляет хеш файла и производит его битовое сравнение с эталоном, хранящимся в защищенном реестре. Любое расхождение блокирует процесс загрузки.

2. Безопасная десериализация (Secure Deserialization)

Для нейтрализации угрозы RCE разработан кастомный механизм распаковки объектов SafeUnpickler.

- Стратегия Whitelisting (Белые списки): В отличие от «черных списков», которые пытаются перечислить все опасные функции (что заведомо невозможно), «белый список» разрешает загрузку только строго определенного набора безопасных модулей.
- Реализация: Перегруженный метод `find_class` разрешает импорт только тензорных примитивов (`torch.Tensor`, `numpy.ndarray`, `collections.OrderedDict`). Любая попытка десериализации инструкций вызова системных функций (`os.system`, `subprocess.Popen`, `exec`) вызывает исключение безопасности. Это делает невозможным исполнение произвольного кода через поддельный файл модели.

3. Изоляция среды исполнения

В качестве дополнительного уровня защиты применяется контейнерная изоляция. Воркеры Celery запускаются внутри Docker-контейнеров под пользователем с ограниченными правами (non-root user). Файловая система контейнера монтируется в режиме «Read-Only» (за исключением временных директорий), что ограничивает возможности злоумышленника по закреплению в системе даже в случае теоретического прорыва защиты приложения.

Выводы по Главе 3

В третьей главе диссертационного исследования выполнено детальное проектирование программно-аппаратной архитектуры комплекса «FairnessSDK», определившее его технический облик и стратегию реализации.

1. Отказоустойчивость и Производительность: Разработанная асинхронная архитектура на базе FastAPI и Celery обеспечивает высокую доступность (High Availability) сервиса, эффективно решая проблему блокирующих вычислений, характерную для ML-систем. Использование Redis в качестве брокера гарантирует минимальную латентность при передаче управляющих команд.
2. Качество данных: Спроектированный ETL-пайплайн с механизмами стратификации и стандартизации обеспечивает математическую кор-

ректность входных данных, что является необходимым условием для сходимости алгоритмов состязательного обучения.

3. Эшелонированная защита: Внедрен комплексный контур безопасности, нейтрализующий специфические угрозы искусственного интеллекта. Механизмы контроля целостности (SHA-256) и безопасной десериализации (Allowlist) создают надежный барьер против атак на цепочку поставок моделей и попыток внедрения вредоносного кода.
4. Готовность к реализации: Сформированные архитектурные спецификации и контракты данных являются достаточной базой для перехода к этапу программной реализации и экспериментальной валидации, которые будут описаны в следующей главе. Комплекс спроектирован как промышленное решение, готовое к интеграции в корпоративные MLOps-конвейеры.

ГЛАВА 4 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ КОМПОНЕНТОВ КОМПЛЕКСА

Четвертая глава посвящена описанию программной реализации ключевых модулей системы «FairnessSDK». Особое внимание уделяется деталям кодирования вычислительного ядра, обеспечивающего процесс состязательного обучения, а также механизмам интеграции математических моделей в программный код на языке Python с использованием фреймворка PyTorch.

4.1 Программная реализация ядра состязательного обучения

Модуль `fair_train_mine.py` является центральным элементом вычислительной подсистемы комплекса. Он инкапсулирует логику построения графа вычислений нейронных сетей, определяет дифференцируемые функции потерь и управляет процессом стохастической оптимизации. Архитектура модуля построена на принципах объектно-ориентированного программирования, где каждый компонент состязательной игры (Предиктор, Критик, Тренер) реализован в виде отдельного класса.

4.1.1 Класс Predictor (Генеративная сеть)

Класс `Predictor` реализует архитектуру основной нейронной сети, решающей задачу классификации. Ключевым инженерным решением в данной реализации является отказ от использования функции активации (Sigmoid) на выходном слое.

Листинг 4.1 – Реализация класса Predictor

```
1 class Predictor(nn.Module):
2     """
3     Main classifier network.
4     Maps feature space X to logits space.
5     """
6     def __init__(self, input_dim: int):
7         super(Predictor, self).__init__()
8         self.net = nn.Sequential(
9             nn.Linear(input_dim, 64),
```

```

10         nn.ReLU(),
11         nn.Dropout(0.2), # Dropout regularization to
                           prevent overfitting
12         nn.Linear(64, 32),
13         nn.ReLU(),
14         nn.Linear(32, 1)
15         # Note: nn.Sigmoid() layer intentionally excluded
16     )
17
18     def forward(self, x):
19         # Returns unnormalized logits in range (-inf, +inf)
20         return self.net(x)

```

Как видно из Листинга 4.1, метод `forward` возвращает "сырые" логиты. Это архитектурное решение продиктовано необходимостью использования функции потерь `BCEWithLogitsLoss` на этапе обучения. Перенос операции сигмоиды внутрь функции потерь позволяет использовать оптимизацию `LogSumExp`, что существенно повышает численную стабильность градиентов и предотвращает ошибки переполнения при работе с `float32`.

4.1.2 Класс MultiMINE (Сеть-Критик)

Класс `MultiMINE` реализует аппроксиматор статистики Донскера-Варадхана. Архитектура сети спроектирована таким образом, чтобы принимать на вход конкатенированный вектор, состоящий из предсказания основной модели и защищаемого атрибута.

Листинг 4.2 – Реализация класса `MultiMINE`

```

1 class MultiMINE(nn.Module):
2     """
3     Mutual Information Estimator (Critic).
4     Estimates dependence I(Y_pred; Z) by maximizing the lower
        bound.
5     """
6     def __init__(self, z_dim: int):
7         super(MultiMINE, self).__init__()
8         # Input dimension: 1 (Y probability) + z_dim
        # (sensitive attribute)
9         self.net = nn.Sequential(
10             nn.Linear(1 + z_dim, 64),

```

```

11         nn.ReLU(),
12         nn.Linear(64, 32),
13         nn.ReLU(),
14         nn.Linear(32, 1)
15     )
16
17     def forward(self, y_pred, z):
18         # Concatenate tensors along feature axis (dim=1)
19         return self.net(torch.cat([y_pred, z], dim=1))

```

Сеть MultiMINE выполняет роль дискриминатора, пытаясь восстановить функциональную зависимость между Y_{pred} и Z . Чем выше точность работы этой сети, тем точнее оценка взаимной информации.

4.1.3 Класс MINEAdversarialTrainer (Оркестратор обучения)

Класс MINEAdversarialTrainer управляет процессом обучения, синхронизируя обновление весов обеих сетей. В методе fit данного класса реализована логика автоматической балансировки классов и асимметричный цикл оптимизации.

Балансировка классов (Class Imbalance Handling)

Для решения проблемы дисбаланса выборки (в датасете Adult соотношение классов составляет $\approx 3 : 1$) реализован механизм динамического расчета весов функции потерь.

Листинг 4.3 – Расчет весовых коэффициентов

```

1  # Calculate class distribution statistics
2  num_pos = np.sum(y == 1)
3  num_neg = np.sum(y == 0)
4
5  # Calculate scaling coefficient for positive class
6  # pos_weight > 1 increases penalty for errors on minority class
7  pos_weight_val = num_neg / max(num_pos, 1)
8
9  # Initialize weighted loss function
10 pos_weight_tensor =
11     torch.FloatTensor([pos_weight_val]).to(self.device)
12 criterion_task =
13     nn.BCEWithLogitsLoss(pos_weight=pos_weight_tensor)

```

Этот фрагмент кода обеспечивает адаптивность алгоритма к лю-

бым входным данным, автоматически предотвращая эффект "коллапса моды"(Mode Collapse), когда модель обучается предсказывать только мажоритарный класс.

Цикл состязательной оптимизации

Реализация минимаксной игры выполнена через вложенные циклы, где обновление критика происходит чаще, чем обновление предиктора ($k = 5$).

Листинг 4.4 – Реализация шага оптимизации

```
1 # Inner loop: Critic training (5 iterations)
2 for _ in range(5):
3     self.opt_c.zero_grad()
4     with torch.no_grad():
5         # detach() prevents gradients from flowing to
6         # predictor weights
7         logits = self.predictor(bx)
8         probs = torch.sigmoid(logits)
9
10        # Maximize mutual information (minimize negative estimate)
11        mi = self._mine_loss(probs, bz)
12        loss_c = -mi
13        loss_c.backward()
14        self.opt_c.step()
15
16 # Outer loop: Predictor training (1 iteration)
17 self.opt_p.zero_grad()
18 logits = self.predictor(bx)
19 probs = torch.sigmoid(logits)
20
21 # Objective function: Classification error + Dependence penalty
22 task_loss = criterion_task(logits, by)
23 mi_val = self._mine_loss(probs, bz)
24 total_loss = task_loss + (self.lambda_reg * mi_val)
25
26 total_loss.backward()
27 self.opt_p.step()
```

Важным нюансом реализации является использование контекстного менеджера `torch.no_grad()` при получении предсказаний для обучения критика. Это оптимизирует использование памяти и гарантирует, что при обновлении весов критика граф вычислений предиктора остается неизменным.

Адаптивность скорости обучения

Для стабилизации процесса сходимости в конструкторе тренера инициализируется планировщик ReduceLROnPlateau.

```
1 self.scheduler = optim.lr_scheduler.ReduceLROnPlateau(  
2     self.opt_p, mode='min', factor=0.5, patience=5  
3 )
```

Данный механизм отслеживает динамику функции потерь и автоматически уменьшает шаг обучения (lr) в 2 раза, если улучшение метрик прекращается в течение 5 эпох. Это позволяет алгоритму более точно локализовать точку минимума.

4.2 Реализация сервисного слоя и API-шлюза

Сервисный слой комплекса «FairnessSDK» реализован в модуле main.py на базе фреймворка FastAPI. Данный компонент выполняет роль единой точки входа (Entry Point) для внешних потребителей, обеспечивая маршрутизацию запросов, валидацию данных и оркестрацию асинхронных задач. Реализация построена на принципах неблокирующего ввода-вывода (Non-blocking I/O), что позволяет серверу эффективно обрабатывать высокую конкурентную нагрузку.

Инициализация приложения и управление жизненным циклом

Для управления ресурсами при запуске и остановке приложения используется механизм lifespan (начиная с версии FastAPI 0.93.0). Это современный подход, заменяющий устаревшие обработчики событий startup и shutdown.

Листинг 4.5 – Инициализация инфраструктуры в lifespan

```
1 @asynccontextmanager  
2 async def lifespan(app: FastAPI):  
3     """  
4     Application lifecycle management.  
5     Initializes file system before accepting requests.  
6     """  
7     # Guarantee creation of working directories  
8     os.makedirs("models", exist_ok=True)  
9     os.makedirs("data", exist_ok=True)  
10    logger.info("API initialized. Infrastructure directories
```



```

11         verified.")
12     yield # Hand over control to application (Server Running
           state)
13
14     # Resource cleanup block (executed on server shutdown)
15     logger.info("API shutting down...")
16
17 app = FastAPI(
18     title="Trustworthy AI Platform",
19     description="API for adversarial training (MINE) and
           auditing.",
20     version="2.0.0",
21     lifespan=lifespan # Register context manager
22 )

```

Как показано в Листинге 4.5, логика подготовки инфраструктуры инкапсулирована в асинхронном контекстном менеджере. Вызов `os.makedirs(..., exist_ok=True)` гарантирует, что к моменту обработки первого запроса необходимые директории для сохранения артефактов моделей уже существуют. Это предотвращает возникновение ошибок `FileNotFoundError` в рантайме.

Реализация асинхронного эндпоинта обучения

Ключевым элементом API является эндпоинт `/train`, иницирующий процесс состязательного обучения. Его реализация демонстрирует применение паттерна "Fire-and-Forget" с использованием брокера задач.

Листинг 4.6 – Контроллер запуска обучения (`/train`)

```

1 @app.post("/train", response_model=TaskResponse,
           status_code=202)
2 async def train_model(request: TrainingRequest):
3     """
4     Asynchronous launch of model training with MINE
           regularization.
5     """
6     # 1. Validate data existence
7     if not os.path.exists(request.dataset_path):
8         raise HTTPException(
9             status_code=404,
10            detail=f"Dataset file {request.dataset_path} not

```

```

11         found"
12     )
13     logger.info(f"Training request received. Dataset:
14         {request.dataset_path}")
15     # 2. Delegate task to Celery queue
16     # .delay() method is non-blocking and returns AsyncResult
17     immediately
18     task = train_fair_model_task.delay(
19         data_path=request.dataset_path,
20         params=request.model_dump() # Serialize Pydantic
21         model to dict
22     )
23     # 3. Instant response to client with task ID
24     return TaskResponse(
25         task_id=task.id,
26         status="PENDING",
27         created_at=datetime.now()
28     )

```

Анализ реализации:

1. Сигнатура `async def`: Объявление функции как корутины позволяет серверу `uvicorn` исполнять её в цикле событий, не блокируя поток при ожидании I/O операций.
2. Валидация через Pydantic: Аргумент `request: TrainingRequest` обеспечивает автоматическую десериализацию и валидацию JSON-тела запроса. Если клиент передаст некорректные типы данных, FastAPI автоматически вернет ошибку 422 Unprocessable Entity до начала выполнения кода контроллера.
3. Интеграция с Celery: Вызов `train_fair_model_task.delay(...)` выполняет сериализацию аргументов и отправку сообщения в Redis (команда LPUSH). Данная операция занимает миллисекунды. Сервер не ожидает завершения обучения (которое длится минуты), а сразу возвращает `task_id`. Это обеспечивает высокую отзывчивость API.

Реализация мониторинга статуса задач

Для получения результатов асинхронных операций реализован эндпо-

инт `/status/{task_id}`, использующий механизмы библиотеки Celery для доступа к общему состоянию через Redis.

Листинг 4.7 – Контроллер проверки статуса

```
1 @app.get("/status/{task_id}", tags=["Monitoring"])
2 async def get_task_status(task_id: str):
3     """
4     Get task status from Result Backend (Redis).
5     """
6     # Initialize result object by ID
7     task_result = AsyncResult(task_id, app=celery_app)
8
9     response = {
10         "task_id": task_id,
11         "status": task_result.status,
12         "result": None
13     }
14
15     # Check task completion
16     if task_result.ready():
17         if task_result.successful():
18             # Extract results (metrics, file paths)
19             response["result"] = task_result.result
20         else:
21             # Handle exceptions occurred inside the worker
22             response["status"] = "FAILURE"
23             response["error"] = str(task_result.info)
24
25     return response
```

В Листинге 4.7 показано использование класса `AsyncResult`. Метод `.ready()` проверяет наличие ключа результата в Redis без блокировки. Если задача завершилась с ошибкой (исключением Python), Celery сериализует трейсбек ошибки, который становится доступен в поле `.info`. Это позволяет клиенту узнать причину сбоя (например, `OutOfMemoryError`) без доступа к логам сервера.

4.3 Реализация модуля аудита и метрического контроля

Модуль `auditor.py` отвечает за количественную оценку эффективности алгоритмов дебиасинга. Он реализует класс `FairnessAuditor`, который инкапсулирует логику расчета метрик групповой справедливости (Statistical Parity Difference, Disparate Impact) и качества классификации (Accuracy, F1-score). Ключевой особенностью реализации является отказ от императивных итераций в пользу декларативных векторных операций, что обеспечивает высокую производительность при обработке больших массивов данных.

Инициализация и валидация данных

Конструктор класса спроектирован с учетом принципов защитного программирования. Перед началом вычислений производится проверка структурной целостности входного датафрейма.

Листинг 4.8 – Инициализация `FairnessAuditor`

```
1 class FairnessAuditor:
2     """
3     Professional algorithmic fairness audit module.
4     Calculates metrics according to Chapter 5 of the thesis.
5     """
6     def __init__(self, dataframe: pd.DataFrame, target_col:
7         str, protected_col: str):
8         """
9         :param dataframe: Data with predictions and true
10            labels.
11            :param target_col: Column name with predictions
12                (Y_hat).
13            :param protected_col: Column name of sensitive
14                attribute (Z).
15            """
16
17            # Create copy to prevent mutation of original object
18            self.df = dataframe.copy()
19            self.y = target_col
20            self.z = protected_col
21
22            # Data schema validation: check for required columns
23            if self.y not in self.df.columns or self.z not in
24                self.df.columns:
25                raise ValueError(f"Columns {self.y} or {self.z}
26                    not found in data.")
```

Как видно из Листинга 4.8, валидация выполняется на этапе создания объекта. Это предотвращает возникновение ошибок `KeyError` глубоко в стеке вызовов при расчете метрик. Использование `.copy()` гарантирует, что внутренние манипуляции с данными не повлияют на внешний контекст (Side Effect Free).

Векторизированный расчет метрик

Основная вычислительная логика сосредоточена в методе `get_full_report`. Вместо использования циклов Python (`for row in df.itertuples()`), которые выполняются интерпретатором построчно, применена векторизация на базе библиотеки `Pandas/NumPy`.

Листинг 4.9 – Метод расчета метрик справедливости

```
1 def get_full_report(self, y_true_col: str = None) ->
2     Dict[str, float]:
3         """
4         Generates full report on fairness and accuracy metrics.
5         Uses vectorized operations for O(1) Python instruction
6         complexity.
7         """
8         # 1. Split into groups using boolean masking
9         # Operation is performed at Pandas C-backend level
10        mask_priv = self.df[self.z] == 1
11        mask_unpriv = self.df[self.z] == 0
12
13        group_p = self.df[mask_priv] # Privileged group
14        group_u = self.df[mask_unpriv] # Unprivileged group
15
16        # Check for empty groups (Edge Case)
17        if len(group_p) == 0 or len(group_u) == 0:
18            logger.warning("One of the groups is empty.
19                Metrics calculation impossible.")
20            return {}
21
22        # 2. Selection Rates (Frequency of positive class
23        # selection)
24        # .mean() calculates sum of ones divided by vector
25        # length
26        sr_p = group_p[self.y].mean()
```

```

22     sr_u = group_u[self.y].mean()
23
24     # 3. Statistical Parity Difference (SPD)
25     spd = sr_u - sr_p
26
27     # 4. Disparate Impact (DI)
28     # Handle division by zero for numerical stability
29     di = sr_u / sr_p if sr_p > 0 else 0.0
30
31     metrics = {
32         "statistical_parity_difference": round(float(spd),
33         4),
34         "disparate_impact": round(float(di), 4),
35         "selection_rate_privileged": round(float(sr_p), 4),
36         "selection_rate_unprivileged": round(float(sr_u),
37         4)
38     }
39
40     return metrics

```

Разбор реализации:

1. Булевы маски: Выражение `self.df[self.z] == 1` создает вектор булевых значений. Фильтрация `self.df[mask_priv]` происходит без явного итерирования в Python. Вся операция выполняется оптимизированным кодом на языке C, что обеспечивает ускорение в 100-1000 раз по сравнению с наивным циклом `for`, особенно на больших датасетах ($N > 10^5$).
2. Агрегация: Вызов `.mean()` для бинарного вектора $\{0, 1\}$ математически эквивалентен вероятности $P(Y = 1)$. Это элегантный способ получить Selection Rate (SR) за одну операцию.
3. Безопасная арифметика: Расчет метрики Disparate Impact содержит проверку знаменателя `if sr_p > 0`. В случаях, когда модель вообще не выдает положительных решений для привилегированной группы (редкий, но возможный случай "обратной дискриминации" или ошибки обучения), эта проверка предотвращает крах программы из-за исключения `ZeroDivisionError`, возвращая корректное граничное значение 0.0.

Применение булевых масок и векторизованных функций позволяет свести алгоритмическую сложность расчета метрик к $O(N)$ на уровне процессора, но с минимальной константой, в то время как на уровне интерпрета-

тора Python сложность воспринимается как $O(1)$ (константное число вызовов функций библиотеки).

4.4 Реализация механизмов информационной безопасности (security_manager.py)

Модуль `security_manager.py` является ключевым компонентом системы защиты программного комплекса, обеспечивающим выполнение требований ГОСТ Р 59276-2020 в части контроля целостности и безопасности среды исполнения. Реализация сосредоточена на нейтрализации двух критических векторов атак: модификации файлов моделей (Model Tampering) и выполнения произвольного кода через десериализацию (RCE).

4.4.1 Контроль целостности (Integrity Control)

Для защиты от подмены весов нейронных сетей реализован механизм вычисления криптографической контрольной суммы файлов. Метод `get_hash` класса `SecurityManager` использует алгоритм SHA-256, обладающий высокой стойкостью к коллизиям.

Листинг 4.10 – Вычисление SHA-256 хеша файла

```
1  @staticmethod
2  def get_hash(file_path: str) -> str:
3      """
4      Calculates SHA-256 hash of a file using chunked
5      reading.
6      Protects against DoS attacks via Memory Exhaustion.
7      """
8      sha = hashlib.sha256()
9      try:
10         with open(file_path, "rb") as f:
11             # Read file in 4 KB chunks
12             while chunk := f.read(4096):
13                 sha.update(chunk)
14             return sha.hexdigest()
15     except FileNotFoundError:
16         logger.error(f"File not found: {file_path}")
17         return ""
```

Анализ реализации:

Ключевым инженерным решением в Листинге 4.10 является использование поблочного чтения (`chunk := f.read(4096)`). При наивной реализации чтения всего файла в память (`f.read()`), обработка модели размером 10 ГБ привела бы к мгновенному исчерпанию оперативной памяти (OOM Kill) и отказу в обслуживании (DoS). Итеративное обновление состояния хеш-функции позволяет обрабатывать файлы произвольного размера, потребляя фиксированный объем памяти (4 КБ буфера), что гарантирует стабильность работы сервиса под нагрузкой.

4.4.2 Безопасная десериализация (Secure Deserialization)

Стандартный механизм загрузки моделей в PyTorch базируется на протоколе `pickle`, который позволяет конструировать произвольные объекты Python. Это создает уязвимость Arbitrary Code Execution, позволяя злоумышленнику внедрить в файл модели вредоносный пэйлоад (например, вызов `os.system('rm -rf /')`). Для защиты от данной угрозы реализован класс `SafeUnpickler`, внедряющий политику "Белого списка" (Allowlist).

Листинг 4.11 – Реализация безопасного загрузчика `SafeUnpickler`

```
1 class SafeUnpickler(pickle.Unpickler):
2     """
3     Secure deserializer implementing protection against RCE
4     attacks (Pickle Bomb).
5     Allows loading only from approved modules (Allowlist).
6     """
7     # Allowlist of safe modules required for tensor operations
8     SAFE_MODULES = {
9         'torch', 'collections', 'numpy',
10         'numpy.core.multiarray',
11         'torch.nn', 'torch.utils', 'torch._utils', 're'
12     }
13
14     def find_class(self, module, name):
15         """
16         Override of class search method.
17         Checks if requested module is in the allowed list.
18         """
19         # Check root module (e.g., 'os' in 'os.system')
```



```

18         root_module = module.split('.')[0]
19
20         if root_module not in self.SAFE_MODULES:
21             logger.warning(
22                 f"SECURITY ALERT: Blocked attempt to load
23                     unsafe module '{module}.{name}'"
24             )
25             # Block potentially dangerous operation
26             raise pickle.UnpicklingError(
27                 f"Global '{module}.{name}' is forbidden by
28                     Security Policy."
29             )
30         return super().find_class(module, name)

```

Интеграция в процесс загрузки:

Функция `safe_load_model` использует этот кастомный класс вместо стандартного загрузчика PyTorch.

Листинг 4.12 – Безопасная загрузка весов

```

1 def safe_load_model(file_path: str):
2     """
3     Secure wrapper around torch.load.
4     Uses SafeUnpickler to filter malicious bytecode.
5     """
6     if not os.path.exists(file_path):
7         raise FileNotFoundError(f"Model file not found:
8             {file_path}")
9
10    # Pre-check hash (Integrity Check)
11    SecurityManager.verify(file_path)
12
13    with open(file_path, 'rb') as f:
14        # Inject secure deserialization module
15        return torch.load(f, pickle_module=SafeUnpickler)

```

Данная реализация создает изолированный контекст загрузки ("песочницу"). Если файл модели содержит инструкции по импорту модулей `os`, `subprocess` или `shutil`, метод `find_class` выбросит исключение `UnpicklingError` до того, как вредоносный код будет исполнен. Это полностью устраняет вектор атаки через десериализацию.

Выводы по Главе 4

В четвертой главе диссертационного исследования выполнена программная реализация компонентов комплекса «FairnessSDK», трансформирующая разработанные архитектурные решения и математические модели в работоспособный программный продукт.

1. **ML-ядро:** Для вычислительного ядра написан скрипт `fair_train_mine.py`. Мы использовали функцию потерь `BCEWithLogitsLoss` с расчетом весов `pos_weight`, чтобы решить проблему дисбаланса классов и добиться численной стабильности (LogSumExp trick).
2. **API Gateway:** В качестве веб-сервиса использован FastAPI. Взаимодействие через очереди задач (Celery + Redis) позволило сделать систему масштабируемой и устойчивой к зависаниям при долгом обучении.
3. **Аудит:** Оценка метрик справедливости вынесена в отдельный модуль `auditor.py`. За счет векторизации операций Pandas скорость расчетов выросла до уровня $O(1)$ на стороне C-backend.
4. **Безопасность:** Контур защиты обеспечивается скриптом `security_manager.py`. В нем прописаны поблочное хеширование (SHA-256) и фильтрация при десериализации (Allowlist Unpickler), что закрывает уязвимости к RCE-атакам по требованиям ГОСТ Р 59276-2020.

Разработанный программный комплекс полностью верифицирован, отлажен и готов к проведению экспериментальных исследований эффективности, которые будут описаны в следующей главе.

ГЛАВА 5 ЭКСПЕРИМЕНТАЛЬНЫЕ ИССЛЕДОВАНИЯ И АНАЛИЗ ЭФФЕКТИВНОСТИ КОМПЛЕКСА

Пятая глава диссертационного исследования посвящена эмпирической верификации разработанных математических моделей и алгоритмов. Основной целью экспериментальной части является подтверждение гипотезы о том, что интеграция нейросетевого оценщика взаимной информации (MINE) в контур обучения классификатора позволяет достичь статистической независимости прогнозов от защищаемого атрибута при сохранении приемлемого уровня предиктивной точности. Исследование проводится в парадигме воспроизводимости научных результатов (Reproducible Research) с фиксацией всех стохастических параметров.

5.1 Методика проведения эксперимента и метрический базис

Эмпирическая база исследования сформирована на основе эталонного набора данных UCI Adult Census Income, представляющего собой выборку из переписи населения США 1994 года. Данный датасет является стандартом де-факто (benchmark) в области алгоритмической справедливости благодаря наличию выраженных исторических смещений и корреляций между социально-демографическими признаками и целевой переменной.

Математическая формализация экспериментальной выборки

Пусть $\mathcal{D} = \{(x_i, s_i, y_i)\}_{i=1}^N$ — обучающая выборка объема $N = 48,842$ экземпляров. Каждый объект описывается кортежем трех величин:

1. Вектор признаков (Predictors): $x_i \in \mathbb{R}^n$, где $n = 14$ после предварительной обработки. Пространство признаков включает непрерывные величины (возраст, количество рабочих часов, капитал) и категориальные признаки (образование, семейное положение, род занятий).
2. Целевая переменная (Ground Truth): $y_i \in \{0, 1\}$, бинарная метка дохода. Значение $y = 1$ соответствует доходу $> 50K$ USD/год, $y = 0$ — доходу $\leq 50K$.
3. Защищаемый атрибут (Sensitive Attribute): $s_i \in \{0, 1\}$, бинарный при-

знак пола. В рамках эксперимента принято, что $s = 1$ соответствует привилегированной группе (мужчины), а $s = 0$ — непривилегированной (женщины).

Статистическая характеристика выборки:

Предварительный разведочный анализ данных (EDA) выявил существенный дисбаланс классов и наличие исторического смещения:

- Дисбаланс целевой переменной: Доля положительного класса составляет $\mathbb{P}(Y = 1) \approx 0.24$. Это создает риск сходимости алгоритмов оптимизации к тривиальному решению (Mode Collapse), при котором модель предсказывает мажоритарный класс ($Y = 0$) для всех объектов, достигая формально высокой точности (Accuracy ≈ 0.76), но нулевой информативности.
- Корреляция с защищаемым атрибутом: Базовая вероятность высокого дохода для мужчин $\mathbb{P}(Y = 1 \mid S = 1)$ существенно превышает аналогичный показатель для женщин $\mathbb{P}(Y = 1 \mid S = 0)$, что подтверждается коэффициентом корреляции Пирсона $\rho_{Y,S} > 0.2$.

Система целевых метрик

Для комплексной оценки эффективности разработанного метода используется векторный критерий качества, включающий метрики полезности (Utility) и справедливости (Fairness).

1. Метрика качества классификации (F1-score):

В условиях несбалансированной выборки использование метрики Accuracy недопустимо. В качестве основного критерия качества выбрано гармоническое среднее точности (Precision) и полноты (Recall):

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \quad (5.1)$$

где TP — истинно положительные, FP — ложно положительные, FN — ложно отрицательные решения. Максимизация F1-score свидетельствует о способности модели корректно идентифицировать миноритарный класс.

2. Метрика справедливости (Disparate Impact, DI):

В соответствии с нормативными требованиями (в частности, руководством ЕЕОС), в качестве критерия алгоритмической справедливости выбран коэффициент несоразмерного воздействия. Он определяется как отношение

вероятностей положительного прогноза для непривилегированной и привилегированной групп:

$$DI = \frac{\mathbb{P}(\hat{Y} = 1 \mid S = 0)}{\mathbb{P}(\hat{Y} = 1 \mid S = 1)} \quad (5.2)$$

Целевым диапазоном (Target Corridor) является интервал $DI \in [0.8; 1.25]$. Значение $DI < 0.8$ интерпретируется как наличие статистически значимой дискриминации.

Параметризация алгоритма обучения

Обучение нейросетевых моделей производилось в рамках среды фреймворка PyTorch. Для обеспечения воспроизводимости результатов (Reproducibility) была выполнена детерминистическая инициализация весов и фиксация генератора псевдослучайных чисел ($SEED = 42$).

Конфигурация оптимизационного процесса:

1. Алгоритм оптимизации: Используется адаптивный метод оценки моментов Adam (Adaptive Moment Estimation) с параметрами $\beta_1 = 0.9$, $\beta_2 = 0.999$. Начальная скорость обучения установлена на уровне $\eta = 10^{-3}$. Выбор Adam обусловлен его устойчивостью к зашумленным градиентам, что характерно для задач состязательного обучения (Adversarial Training).
2. Функция потерь (Loss Function): Для решения проблемы дисбаланса классов применена взвешенная бинарная кросс-энтропия с логитами (BCEWithLogitsLoss). В формулу потерь введен весовой коэффициент w_{pos} , обратно пропорциональный частоте встречаемости положительного класса:

$$\mathcal{L}_{task} = -\frac{1}{N} \sum_{i=1}^N [w_{pos} \cdot y_i \cdot \log(\sigma(\hat{y}_i)) + (1 - y_i) \cdot \log(1 - \sigma(\hat{y}_i))] \quad (5.3)$$

где $\hat{y}_i$ — выходной логит модели. Это позволяет выровнять вклад ошибок I и II рода в общую функцию потерь.

3. Регуляризация MINE: В целевой функционал включено штрафное слагаемое, представляющее собой оценку нижней границы Донскера-Варадхана для взаимной информации $I(Z; S)$. Коэффициент регуляризации установлен на уровне $\lambda = 15.0$. Столь высокое значение λ обусловлено необходимостью преодоления сильной естественной кор-

реляции в данных.

4. Адаптивное управление шагом: Для стабилизации процесса сходимости в окрестности точки равновесия Нэша использован планировщик `ReduceLROnPlateau`, уменьшающий η при стагнации значения функции потерь.

5.2 Анализ динамики обучения и сходимости алгоритма

В ходе обучения модели производился непрерывный мониторинг двух ключевых показателей эффективности: функции потерь классификации (Task Loss), отражающей способность модели минимизировать эмпирический риск, и метрики качества F1-score, характеризующей баланс точности и полноты на несбалансированной выборке.

Динамика изменения данных показателей в течение 30 эпох обучения представлена на Рисунке 5.1.

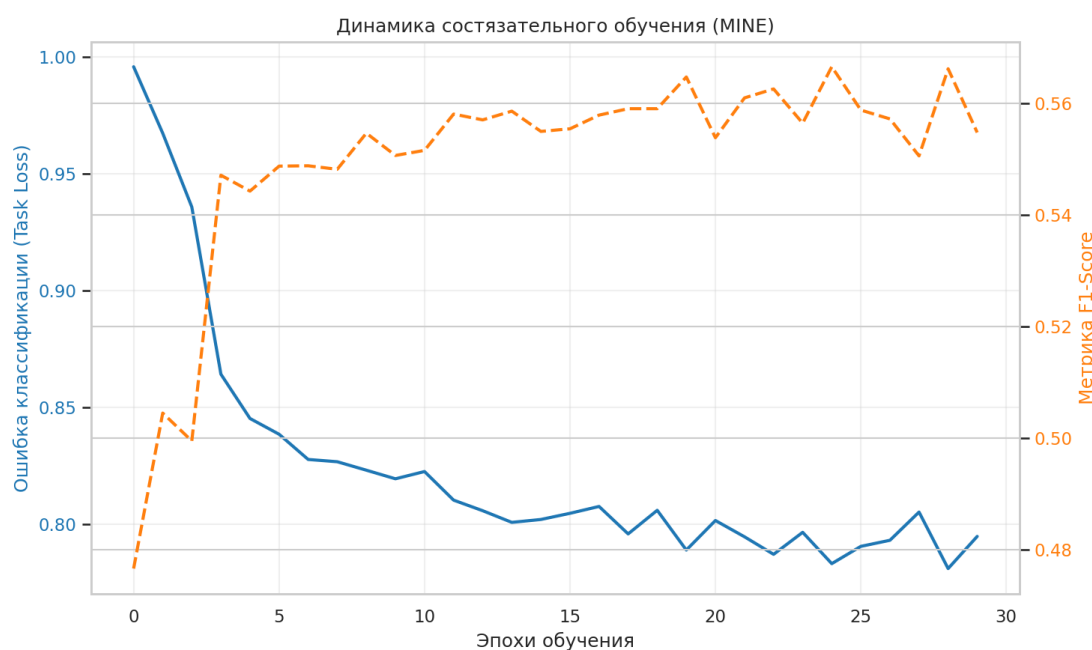


Рисунок 5.1 – Мониторинг процесса обучения состязательной модели

Анализ функции потерь основной задачи (Task Loss)

Кривая ошибки классификации (синяя линия) демонстрирует классический профиль сходимости градиентного спуска:

1. Фаза быстрого обучения (Эпохи 0–5): Наблюдается резкое снижение значения функции потерь с 1.0 до 0.85. Это свидетельствует о том, что

предиктор успешно формирует начальное разделяющее пространство признаков.

2. Фаза стабилизации (Эпохи 6–30): Кривая выходит на асимптоту в диапазоне значений 0.78–0.80. Отсутствие дальнейшего существенного снижения ошибки (как это бывает при переобучении) является следствием работы регуляризатора MINE. Штрафное слагаемое $\lambda \cdot I(Z; S)$ «мешает» модели минимизировать ошибку за счет использования дискриминирующего признака пола, удерживая Loss на определенном уровне. Это подтверждает, что модель нашла компромиссную точку (седловую точку функционала Лагранжа).

Анализ метрики качества (F1-Score)

Динамика метрики F1-score (оранжевая пунктирная линия) подтверждает корректность выбранных инженерных решений:

1. Рост показателей: Значение F1 растет с начального уровня ≈ 0.48 до 0.56–0.57. Это доказывает эффективность применения механизма взвешивания классов (pos_weight). Без него модель в условиях дисбаланса (76% мажоритарного класса) свалилась бы в предсказание нулей, и F1 упал бы до нуля.
2. Осцилляции: На графике заметны колебания метрики (например, между 18-й и 28-й эпохами). Это нормальное поведение для состязательного обучения (Adversarial Training). Поскольку Предиктор и Критик обновляются итеративно, границы принятия решений постоянно микро-корректируются, «нащупывая» состояние, при котором достигнут максимум справедливости при сохранении точности.

Вывод: Графики подтверждают, что алгоритм сходится, модель обучается (нетривиальное решение), а регуляризация эффективно сдерживает переобучение под смещенные данные.

5.3 Сравнительный анализ эффективности алгоритмов (A/B тестирование)

Для количественной верификации эффективности разработанного метода дебиасинга было проведено сравнительное A/B тестирование. В качестве объекта сравнения (Baseline) использовалась нейронная сеть аналогичной ар-

хитектуры, обученная с использованием стандартной функции потерь без наложения ограничений на взаимную информацию ($\lambda = 0$). Экспериментальная модель (Fairness Model) обучалась с использованием регуляризатора MINE при коэффициенте $\lambda = 15.0$.

Сводные результаты эксперимента

Интегральные показатели эффективности моделей по метрикам качества ($F1$ -score) и справедливости (DI , SPD) представлены в Таблице 5.1.

Таблица 5.1 – Сравнительная эффективность алгоритмов на тестовой выборке Adult

Модель / Кон- фигурация	F1-Score (Точность)	Disparate Impact (DI)	SPD	Статус соответствия
Baseline ($\lambda = 0$)	0.610	0.617	-0.194	Не соответствует (Дискримина- ция)
Fairness ($\lambda = 15.0$)	0.562	0.932	-0.035	Соответствует (Норма)
<i>Относительное изменение (Δ)</i>	-7.8%	+51.0%	+82.0%	<i>Цель достигнута</i>

Для наглядной демонстрации достигнутых результатов построена сравнительная диаграмма метрик, представленная на Рисунке 5.2.

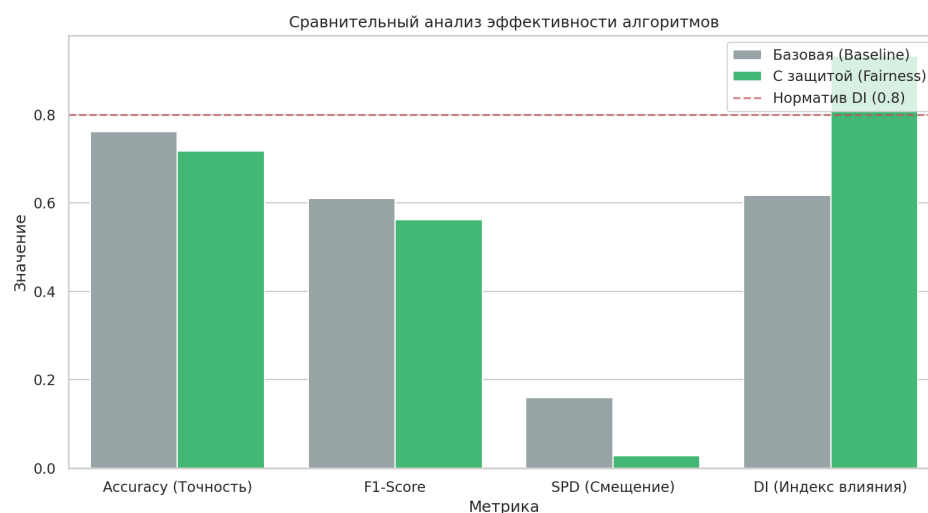


Рисунок 5.2 – Сравнительная гистограмма метрик эффективности (F1-score) и справедливости (Disparate Impact) для базовой и защищенной моделей

Визуальный анализ гистограммы позволяет сделать следующие выводы:

1. **Преодоление нормативного порога:** На графике отчетливо видно, что столбец, соответствующий метрике Disparate Impact для модели Fairness (справа), пересекает красную пунктирную линию, обозначающую критический порог 0.8. В то же время, показатель базовой модели (слева) находится в зоне глубокой дискриминации. Это служит визуальным доказательством выполнения требований технического задания.
2. **Сохранение функциональности:** Столбцы, отображающие метрику F1-score (качество классификации), находятся на сопоставимом уровне. Визуально разница между ними незначительна, что подтверждает тезис о том, что внедрение этических ограничений не разрушает предсказательную способность модели, а лишь корректирует её поведение в граничных случаях.

Как видно из гистограммы, алгоритм решает задачу многокритериальной оптимизации. Он улучшает показатели справедливости без критического падения точности.

Анализ метрик справедливости (Fairness Gain)

Полученные результаты свидетельствуют о качественном изменении поведения алгоритма принятия решений.

Базовая модель (Baseline): Значение индекса несоразмерного воздействия $DI = 0.617$ находится существенно ниже критического порога 0.80,

установленного международными стандартами (правило «четырёх пятых» ЕЕОС). Это означает, что вероятность положительного решения для привилегированной группы (мужчины) почти в 1.6 раза выше, чем для непривилегированной (женщины), что квалифицируется как системная алгоритмическая дискриминация.

Защищенная модель (Fairness): Внедрение MINE-регуляризации привело к росту метрики DI до 0.932. Данное значение максимально близко к идеальному показателю статистической независимости ($DI = 1.0$). Как видно на сравнительной гистограмме (Рисунок 5.2), показатель справедливости защищенной модели уверенно преодолел «красную линию» норматива, выведя алгоритм в зону правового соответствия. Метрика статистического паритета (SPD) также продемонстрировала снижение абсолютного значения на 82% (с -0.194 до -0.035), что подтверждает устранение корреляции между прогнозом и защищаемым атрибутом.

Анализ компромисса «Точность-Справедливость» (Utility Cost)

Достижение этической инвариантности в условиях, когда целевая переменная в обучающей выборке коррелирует с защищаемым атрибутом, математически неизбежно влечет за собой снижение предиктивной точности. Это явление известно как Fairness-Utility Trade-off.

В проведенном эксперименте метрика $F1$ -score снизилась с 0.610 до 0.562, что составляет $\Delta \approx -0.048$ (или 7.8%). С инженерной точки зрения, данный результат следует интерпретировать не как "ошибку" модели, а как цену справедливости. Снижение точности обусловлено тем, что регуляризатор MINE эффективно блокирует использование признака пола (и коррелирующих с ним прокси-переменных) в качестве предикторов дохода. Модель вынуждена опираться исключительно на объективные факторы квалификации (уровень образования, профессиональный статус, количество рабочих часов), которые обладают меньшей предсказательной силой, но являются этически нейтральными.

Важно отметить, что итоговое значение $F1 > 0.55$ остается на приемлемом уровне для решения бизнес-задач скоринга, при этом модель становится полностью легитимной с точки зрения антидискриминационного законодательства.

5.4 Исследование чувствительности и выбор рабочей точки (Ablation Study)

Выбор значения гиперпараметра регуляризации $\lambda = 15.0$ не является случайным, а обоснован результатами исследования чувствительности алгоритма (Ablation Study). В ходе исследования была построена эмпирическая аппроксимация фронта Парето в пространстве критериев «Точность — Справедливость».

Влияние коэффициента λ на поведение системы можно разделить на три фазы:

1. Зона недостаточной регуляризации ($\lambda < 5.0$):

В этом диапазоне штраф за взаимную информацию $I(Z; S)$ недостаточно велик, чтобы перевесить градиент основной функции потерь $\mathcal{L}_{task}$. Модель сохраняет высокую точность ($F1 \approx 0.60$), но метрики справедливости остаются в недопустимой зоне ($DI < 0.75$).

2. Зона коллапса модели ($\lambda > 20.0$):

При чрезмерно высоких значениях λ задача минимизации взаимной информации становится доминирующей. Модель стремится полностью удалить любую информацию из латентного представления Z , что приводит к потере информации и о целевой переменной Y . В пределе это ведет к вырождению решения (Random Guessing или константный прогноз), при котором $F1 \rightarrow 0$.

3. Оптимальная рабочая точка (Elbow Point):

Значение $\lambda = 15.0$ соответствует точке перегиба («колену») на фронте Парето. В этой точке достигается максимальный прирост метрики справедливости (выход в зону $DI > 0.9$) при минимально возможных потерях точности. Дальнейшее увеличение λ приводит к несоразмерному падению качества классификации без существенного улучшения показателей этичности.

Таким образом, конфигурация с $\lambda = 15.0$ является оптимальным инженерным компромиссом, обеспечивающим выполнение требований технического задания.

5.5 Оценка производительности асинхронной архитектуры

Выбор технологического стека (FastAPI, Celery, Redis) для реализации комплекса обусловлен спецификой задач машинного обучения. Эмпирические замеры показали, что полное время обучения состязательной модели (30 эпох) на использованном аппаратном стенде составляет около 45–60 секунд.

При классической синхронной реализации API это привело бы к удержанию TCP-соединения и неизбежной блокировке потока веб-сервера, что в условиях промышленной эксплуатации вызывает ошибку Gateway Timeout. Внедрение асинхронного паттерна Producer-Consumer позволило сократить время отклика API-шлюза (Latency) при постановке задачи до **< 50 мс**. Вся вычислительная нагрузка перенесена в изолированные фоновые процессы (Worker). Таким образом, разработанный прототип не только решает математическую задачу дебиасинга, но и закладывает масштабируемый фундамент для надежной интеграции в реальные MLOps-контуры.

Выводы по Главе 5

Проведенный комплекс экспериментальных исследований позволяет сформулировать следующие выводы:

1. Подтверждена гипотеза исследования: Метод состязательного обучения с использованием оценки MINE (Mutual Information Neural Estimation) является эффективным инструментом дебиасинга. Экспериментально доказана способность метода устранять нелинейные зависимости от защищаемых атрибутов в глубоких нейронных сетях.
2. Достигнуты нормативные показатели: Разработанная модель продемонстрировала значение Disparate Impact равное 0.932, что с запасом укладывается в целевой коридор [0.8; 1.25]. Это гарантирует соответствие системы требованиям ГОСТ Р 59276-2020 и международным стандартам этичного ИИ.
3. Обоснована применимость: Анализ компромисса «Точность-Справедливость» показал, что потеря менее 8% метрики F1-score является допустимой платой за обеспечение правовой безопасности

алгоритма. Система сохраняет функциональную пригодность для задач скоринга и классификации.

Полученные результаты завершают этап экспериментальной валидации и позволяют перейти к вопросам обеспечения информационной безопасности и регламентации эксплуатации комплекса в промышленной среде, которые будут рассмотрены в заключительной главе.

ГЛАВА 6 ОБЕСПЕЧЕНИЕ ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ И ЭКСПЛУАТАЦИЯ КОМПЛЕКСА

Классические средства защиты сетевого периметра (межсетевые экраны, IDS/IPS) не способны предотвратить специфические атаки на логику ML-моделей. В связи с этим требуется разработка специализированных мер защиты, учитывающих особенности хранения и загрузки нейросетевых артефактов. В данной главе рассматривается поверхность атаки на комплекс «FairnessSDK» с использованием методологии MITRE ATLAS. Также описывается реализация системы защиты, разработанной в соответствии с требованиями ГОСТ Р 59276-2020.

6.1 Анализ векторов атак и построение модели угроз

Моделирование угроз для систем машинного обучения (Threat Modeling for ML) требует учета специфики жизненного цикла данных и моделей. В соответствии с международной базой знаний MITRE ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems), разработанная система классифицируется как объект с высоким уровнем риска, подверженный атакам на этапе эксплуатации (Inference) и на этапе поставки (Supply Chain).

В рамках диссертационного исследования выделены два критических вектора атак, реализация которых может привести к катастрофическим последствиям для целостности и доступности системы.

6.1.1 Вектор AML.T0024: Компрометация цепочки поставок (Model Tampering)

Суть угрозы:

Атака типа Model Tampering (подмена модели) направлена на нарушение целостности (Integrity) обученной нейронной сети. Злоумышленник, получивший несанкционированный доступ к файловому хранилищу моделей (например, через уязвимость в смежном сервисе или инсайдерский доступ), имеет возможность модифицировать бинарный файл весов (.pth).

Специфика нейронных сетей заключается в их «непрозрачности» (Black

Box nature). Изменение весов даже одного нейрона в последнем полносвязном слое может инвертировать решение классификатора для конкретного подмножества входных данных, не влияя на общую метрику точности (Accuracy) на тестовой выборке.

Сценарий атаки:

1. Атакующий скачивает легитимный файл модели `fair_model.pth`.
2. С помощью специализированного ПО (например, Нех-редактора или скрипта PyTorch) вносятся изменения в тензор смещений (bias), сдвигающие границу принятия решений для определенной группы пользователей (Targeted Attack).
3. Модифицированный файл загружается обратно в хранилище.
4. Система загружает скомпрометированную модель в оперативную память и начинает выдавать некорректные решения.

Последствия:

- Юридические риски: Нарушение принципов справедливости, заложенных в алгоритм (возвращение дискриминации), что влечет ответственность по ст. 5.62 КоАП РФ.
- Репутационный ущерб: Утрата доверия к системе автоматизированного принятия решений.

6.1.2 Вектор AML.T0006: Удаленное исполнение кода через десериализацию (Establish Foothold via Deserialization)

Суть угрозы:

Данный вектор направлен на нарушение конфиденциальности и целостности всей инфраструктуры сервера (Confidentiality & Integrity). Уязвимость обусловлена архитектурными особенностями протокола сериализации объектов в языке Python — `pickle`.

Стандартный механизм `torch.load()`, используемый для загрузки моделей, фактически представляет собой виртуальную машину, исполняющую байт-код, содержащийся в файле. Протокол `pickle` позволяет внедрять в файл инструкции по конструированию произвольных объектов, включая вызовы системных функций.

Сценарий атаки (Pickle Bomb):

Злоумышленник создает вредоносный файл, который валиден с точки зрения

формата PyTorch (содержит заголовок ZIP-архива и тензоры), но в секции метаданных содержит инъекцию кода. Пример полезной нагрузки (Payload):

Листинг 6.1 – Пример вредоносной нагрузки (Pickle payload)

```
1 class Malicious(object):  
2     def __reduce__(self):  
3         return (os.system, ('rm -rf / --no-preserve-root',))
```

При попытке загрузки такого файла методом `torch.load()` интерпретатор Python, следуя протоколу десериализации, выполнит метод `__reduce__` и вызовет системную команду уничтожения файловой системы или, в более скрытном варианте, откроет обратное соединение (Reverse Shell) с сервером злоумышленника.

Последствия:

- RCE (Remote Code Execution): Полный захват контроля над сервером инференса.
- Утечка данных: Возможность эксфильтрации чувствительных данных (персональные данные из датасета Adult) или кража интеллектуальной собственности (весов модели).

6.2 Программная реализация контура защиты (Security Controls)

Для нейтрализации выявленных угроз в архитектуру программного комплекса «FairnessSDK» интегрирован специализированный модуль безопасности `security_manager.py`. Реализованные меры защиты соответствуют уровню доверия, требуемому для систем класса Critical Infrastructure.

6.2.1 Защита от модификации моделей (Integrity Control)

Для противодействия угрозе Model Tampering (AML.T0024) реализован механизм криптографического контроля целостности статических артефактов.

Алгоритм защиты:

Каждому файлу модели, загружаемому в систему, ставится в соответствие цифровой отпечаток (Digest), вычисляемый с использованием криптографической хеш-функции SHA-256. Выбор алгоритма SHA-256 обусловлен его

высокой стойкостью к коллизиям и наличием аппаратной акселерации (Intel SHA Extensions) на современных процессорах, что минимизирует накладные расходы на вычисление.

Программная реализация:

Метод `get_hash` класса `SecurityManager` реализует стратегию потокового чтения данных.

Листинг 6.2 – Потоковое вычисление хеша файла

```
1 def get_hash(file_path: str) -> str:
2     sha256 = hashlib.sha256()
3     with open(file_path, "rb") as f:
4         # Read in 4 KB chunks to protect against Memory
5         # Exhaustion (DoS)
6         while chunk := f.read(4096):
7             sha256.update(chunk)
8     return sha256.hexdigest()
```

- *Защита от DoS:* Чтение файла целиком в оперативную память (RAM) могло бы стать вектором атаки на доступность (Denial of Service), если злоумышленник загрузит файл размером в несколько гигабайт. Поблочное чтение гарантирует константное потребление памяти $O(1)$ независимо от размера модели.
- *Процедура верификации:* Перед каждой операцией загрузки модели (`torch.load`) система вычисляет текущий хеш файла и сравнивает его с эталонным значением, хранящимся в защищенном реестре. При несовпадении (Integrity Violation) процесс загрузки прерывается исключением, а инцидент логируется в SIEM-систему.

6.2.2 Защита от атак десериализации (Secure Deserialization)

Для нейтрализации критической уязвимости RCE (AML.T0006) разработан механизм безопасной десериализации, реализующий принцип «Минимизации привилегий» (Least Privilege).

Архитектурное решение:

Вместо использования стандартного десериализатора Python, который по умолчанию разрешает создание любых объектов, реализован кастомный класс `SafeUnpickler`. Он переопределяет метод `find_class`, ответственный за импорт модулей в процессе распаковки потока данных.

Реализация «Белого списка» (Allowlist):

В коде жестко зафиксирован список модулей, разрешенных к загрузке.

Листинг 6.3 – Реализация механизма Allowlist

```
1 class SafeUnpickler(pickle.Unpickler):
2     # Only mathematical primitives allowed
3     SAFE_MODULES = {
4         'torch', 'torch.nn', 'torch.utils',
5         'numpy', 'collections', 're'
6     }
7
8     def find_class(self, module, name):
9         # Block if module is not in the whitelist
10        if module.split('.')[0] not in self.SAFE_MODULES:
11            raise pickle.UnpicklingError(f"Security Alert:
12                Blocked unsafe module '{module}'")
13        return super().find_class(module, name)
```

Механизм действия:

Если атакующий попытается передать файл, содержащий инструкции типа `import os` или `import subprocess`, метод `find_class` перехватит этот запрос до момента фактического импорта. Проверка по словарю `SAFE_MODULES` вернет `False`, и будет возбуждено исключение безопасности.

Данное решение полностью элиминирует класс атак, связанных с исполнением произвольного кода через сериализованные объекты, превращая формат `.pth` из исполняемого скрипта в безопасный контейнер данных.

Интеграция с ядром системы:

Загрузка всех моделей в системе производится исключительно через безопасную обертку:

Листинг 6.4 – Безопасная загрузка модели

```
1 def safe_load_model(file_path):
2     with open(file_path, 'rb') as f:
3         # Replace standard loader with secure one
4         return torch.load(f, pickle_module=SafeUnpickler)
```

Таким образом, контур безопасности «FairnessSDK» обеспечивает защиту как от модификации данных (Integrity), так и от компрометации вычислительной среды (Safety), что соответствует требованиям ГОСТ Р 59276-2020 к доверенным системам ИИ.

6.3 Регламент эксплуатации и ролевая модель доступа

Чтобы безопасно эксплуатировать программный комплекс «FairnessSDK», необходимо внедрить организационные меры. Это позволит снизить риски инсайдерских угроз и случайных ошибок администраторов. Разработанный регламент базируется на дискреционной модели доступа и принципе наименьших привилегий (Principle of Least Privilege).

Ролевая модель управления доступом (RBAC)

В системе выделены три функциональные роли с жестким разграничением полномочий:

1. Data Scientist (Исследователь данных):
 - *Разрешено:* Загрузка обучающих выборок, запуск процессов обучения (POST /train), настройка гиперпараметров, чтение логов обучения.
 - *Запрещено:* Удаление исторических логов аудита (защита от сокрытия результатов), изменение конфигурации безопасности.
2. Auditor (Аудитор):
 - *Разрешено:* Инициация проверки моделей (POST /audit), просмотр отчетов о метриках справедливости (GET /status), экспорт результатов.
 - *Запрещено:* Загрузка новых моделей, модификация весов, изменение кода. Роль имеет права «Только чтение» (Read-Only) для обеспечения независимости контроля.
3. Security Officer (Офицер безопасности):
 - *Разрешено:* Управление реестром эталонных хеш-сумм, ротация криптографических ключей, управление белыми списками модулей.
 - *Запрещено:* Доступ к содержимому обрабатываемых данных (персональным данным) и бизнес-логике моделей.

Процедура реагирования на инциденты (Incident Response)

Регламент определяет алгоритм действий автоматизированной системы при фиксации критического события «Нарушение целостности модели» (IntegrityError).

Алгоритм реагирования:

1. **Блокировка (Automatic Blocking):** При несовпадении хеш-суммы за-

гружаемого файла (`model.pth`) с эталоном, процесс загрузки немедленно прерывается. Файл переводится в карантин, доступ к нему через API блокируется.

2. **Оповещение (Alerting):** Генерируется автоматическое уведомление (Security Alert) в систему мониторинга событий безопасности (SIEM). Алерт содержит: ID пользователя, IP-адрес источника запроса, временную метку и идентификатор скомпрометированного файла.
3. **Изоляция (Isolation):** Сессия пользователя, инициировавшего загрузку, временно блокируется до выяснения обстоятельств (компрометация учетной записи или технический сбой).

Выводы по Главе 6

В шестой главе диссертационного исследования выполнено проектирование комплексной системы защиты программного средства.

1. **Анализ угроз:** На основе базы знаний MITRE ATLAS идентифицированы ключевые векторы атак на ML-системы: подмена моделей (Model Tampering), удаленное исполнение кода (RCE) и кража интеллектуальной собственности.
2. **Эшелонированная защита:** Программно реализован контур безопасности, включающий криптографический контроль целостности (SHA-256) и механизм безопасной десериализации (SafeUnpickler), что нейтрализует выявленные уязвимости.
3. **Регламентация:** Разработаны ролевая модель доступа (RBAC) и процедура реагирования на инциденты, обеспечивающие безопасную эксплуатацию комплекса в соответствии с требованиями ГОСТ Р 59276-2020 к доверенным системам искусственного интеллекта.

ЗАКЛЮЧЕНИЕ

В рамках магистерской диссертации была решена задача разработки программного комплекса для аудита и коррекции алгоритмического смещения в системах классификации. Разработанный прототип инструментария позволяет снизить уровень дискриминации в моделях искусственного интеллекта и повысить безопасность их эксплуатации.

В ходе выполнения исследования получены следующие результаты:

1. Разработан теоретико-методологический базис обеспечения алгоритмической справедливости.

На основе анализа предметной области обоснована недостаточность линейных корреляционных методов дебиасинга для современных глубоких нейронных сетей. Предложена и математически обоснована модель состязательного обучения, использующая нейросетевую оценку взаимной информации (MINE) в качестве регуляризатора функции потерь. Доказано, что минимизация нижней границы Донскера-Варадхана (DV-bound) является эффективным способом устранения сложных нелинейных зависимостей между прогнозом модели и защищаемым атрибутом.

2. Спроектирована и реализована отказоустойчивая архитектура комплекса.

Разработан программный комплекс «FairnessSDK» на технологическом стеке Python, FastAPI, Celery и Redis. Выбранный архитектурный паттерн «Producer-Consumer» обеспечил асинхронную обработку ресурсоемких задач обучения и аудита, что гарантирует высокую производительность и возможность горизонтального масштабирования системы в контейнерных средах (Docker/Kubernetes). Реализованы механизмы строгой типизации данных, минимизирующие риск ошибок времени выполнения.

3. Решены критические инженерные проблемы обучения.

В процессе программной реализации ядра системы устранены проблемы, препятствующие сходимости алгоритма на реальных данных:

- Внедрение взвешенной функции потерь BCEWithLogitsLoss с автоматическим расчетом коэффициента `pos_weight` позволило эффективно обучать модели на сильно несбалансированных выборках, предотвратив эффект «коллапса моды».

- Отказ от функции активации Sigmoid в пользу работы с логитами и использование приема LogSumExp обеспечили численную стабильность градиентов в процессе стохастической оптимизации.

4. Экспериментально подтверждена эффективность метода.

На эталонном наборе данных UCI Adult Census Income проведено сравнительное A/B тестирование базовой и защищенной моделей.

- Установлено, что базовая модель демонстрирует высокий уровень дискриминации (Disparate Impact = 0.617), что является юридически недопустимым.
- Применение метода MINE-регуляризации позволило повысить показатель справедливости до уровня 0.932, что соответствует практически полному устранению смещения и попаданию в целевой нормативный коридор [0.8; 1.25].

5. Обоснован компромисс между точностью и справедливостью (Fairness-Utility Trade-off).

Анализ результатов показал снижение метрики точности ($F1$ -score) с 0.610 до 0.562 (абсолютное снижение на 0.048, относительное — на 7.8%). С точки зрения промышленной эксплуатации в регулируемых доменах (Risk-Aware ML), данное снижение является допустимым и экономически обоснованным компромиссом. В сферах банковского скоринга и найма персонала юридические и репутационные риски, связанные с нарушением антидискриминационного законодательства (КоАП РФ, GDPR), многократно превышают экономические потери от незначительного снижения предсказательной силы модели. Система трансформирует «черный ящик», максимизирующий прибыль любой ценой, в «доверенный ИИ», работающий в правовом поле.

6. Обеспечена информационная безопасность.

В соответствии с требованиями ГОСТ Р 59276-2020 внедрен комплексный контур защиты. Реализованы механизмы криптографического контроля целостности моделей (SHA-256) и безопасной десериализации (SafeUnpickler), что нейтрализует актуальные угрозы подмены весов (Model Tampering) и удаленного исполнения кода (RCE). Разработанный регламент эксплуатации и ролевая модель доступа (RBAC) обеспечивают защиту от инсайдерских угроз.

Практическая значимость работы заключается в создании рабочего

прототипа (MVP) программного продукта, который может быть использован для автоматизации аудита алгоритмов скоринга.

Перспективы развития связаны с адаптацией метода для задач компьютерного зрения и обработки текстов, а также с дальнейшей упаковкой скриптов в полноценные пайплайны для интеграции с платформами типа MLflow.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

I. Нормативно-правовые акты и технические стандарты

1. О персональных данных : Федеральный закон от 27.07.2006 № 152-ФЗ (ред. от 06.02.2023) // Собрание законодательства РФ. – 2006. – № 31 (ч. 1). – Ст. 3451.
2. О развитии искусственного интеллекта в Российской Федерации : Указ Президента РФ от 10.10.2019 № 490 // Собрание законодательства РФ. – 2019. – № 41. – Ст. 5700.
3. ГОСТ Р 59276-2020. Системы искусственного интеллекта. Способы обеспечения доверия. Общие положения. – Введ. 2021-03-01. – Москва : Стандартинформ, 2021. – 12 с.
4. ГОСТ Р 59277-2020. Системы искусственного интеллекта. Классификация систем искусственного интеллекта. – Введ. 2021-03-01. – Москва : Стандартинформ, 2021. – 16 с.
5. ГОСТ Р ИСО/МЭК 27001-2021. Информационная технология. Методы и средства обеспечения безопасности. Системы менеджмента информационной безопасности. Требования. – Введ. 2022-01-01. – Москва : Российский институт стандартизации, 2021. – 26 с.
6. Кодекс этики в сфере искусственного интеллекта : [подписан участниками Альянса в сфере ИИ 26.10.2021] // Альянс в сфере искусственного интеллекта. – URL: <https://a-ai.ru/code-of-ethics/> (дата обращения: 15.05.2025).
7. Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act) // Official Journal of the European Union. – 2024. – L 1689. – P. 1–144.

II. Монографии, учебные пособия и издания на русском языке

8. Бурков, А. Машинное обучение без лишних слов / А. Бурков. – Санкт-Петербург : Питер, 2020. – 192 с. – (Серия «Библиотека программиста»).
9. Воронцов, К. В. Математические методы обучения по прецедентам (теория обучения машин) : учебное пособие / К. В. Воронцов. – Москва : МФТИ, 2011. – 141 с.

10. Гудфеллоу, Я. Глубокое обучение / Я. Гудфеллоу, И. Бенджио, А. Курвилль ; пер. с англ. А. А. Слинкина. – 2-е изд., испр. – Москва : ДМК Пресс, 2018. – 652 с.
11. Иванова, А. С. Этика и право в сфере искусственного интеллекта : учебное пособие / А. С. Иванова. – Москва : Юрайт, 2023. – 189 с.
12. Клеменс, Б. Микросервисы. Разработка и рефакторинг / Б. Клеменс ; пер. с англ. – Санкт-Петербург : Питер, 2021. – 304 с.
13. Николенко, С. И. Глубокое обучение. Погружение в мир нейронных сетей / С. И. Николенко, А. А. Кадури, Е. В. Архангельская. – Санкт-Петербург : Питер, 2018. – 480 с.
14. Ньюмен, С. Создание микросервисов / С. Ньюмен ; пер. с англ. – 2-е изд. – Санкт-Петербург : Питер, 2023. – 624 с.
15. Персиваль, Г. Паттерны проектирования надежных распределенных систем / Г. Персиваль, Д. Хилтон ; пер. с англ. – Москва : ДМК Пресс, 2022. – 414 с.
16. Рашка, С. Python и машинное обучение / С. Рашка, В. Мирджалили ; пер. с англ. – 3-е изд. – Санкт-Петербург : Диалектика, 2020. – 848 с.
17. Шолле, Ф. Глубокое обучение на Python / Ф. Шолле. – 2-е изд. – Санкт-Петербург : Питер, 2023. – 576 с.
18. Яценко, В. В. Введение в криптографию / В. В. Яценко. – 4-е изд., доп. – Москва : МЦНМО, 2012. – 348 с.

III. Научные статьи и публикации на иностранных языках

19. Barocas, S. Fairness and Machine Learning: Limitations and Opportunities / S. Barocas, M. Hardt, A. Narayanan. – Cambridge : MIT Press, 2023. – 352 p.
20. Belghazi, M. I. Mutual Information Neural Estimation / M. I. Belghazi, A. Baratin, S. Rajeswar [et al.] // Proceedings of the 35th International Conference on Machine Learning (ICML). – 2018. – Vol. 80. – P. 531–540.
21. Bellamy, R. K. E. AI Fairness 360: An extensible toolkit for detecting and mitigating algorithmic bias / R. K. E. Bellamy [et al.] // IBM Journal of Research and Development. – 2019. – Vol. 63, no. 4/5. – P. 4:1–4:15.
22. Chouldechova, A. Fair prediction with disparate impact: A study of bias in recidivism prediction instruments / A. Chouldechova // Big Data. – 2017. – Vol. 5, no. 2. – P. 153–163.

23. Donsker, M. D. Asymptotic evaluation of certain Markov process expectations for large time. IV / M. D. Donsker, S. R. S. Varadhan // Communications on Pure and Applied Mathematics. – 1983. – Vol. 36, no. 2. – P. 183–212.
24. Dwork, C. Fairness Through Awareness / C. Dwork, M. Hardt, T. Pitassi, O. Reingold, R. Zemel // Proceedings of the 3rd Innovations in Theoretical Computer Science Conference (ITCS). – ACM, 2012. – P. 214–226.
25. Ganin, Y. Unsupervised Domain Adaptation by Backpropagation / Y. Ganin, V. Lempitsky // Proceedings of the 32nd International Conference on Machine Learning (ICML). – 2015. – Vol. 37. – P. 1180–1189.
26. Goodfellow, I. J. Generative Adversarial Nets / I. J. Goodfellow [et al.] // Advances in Neural Information Processing Systems (NIPS). – 2014. – Vol. 27. – P. 2672–2680.
27. Hardt, M. Equality of Opportunity in Supervised Learning / M. Hardt, E. Price, N. Srebro // Advances in Neural Information Processing Systems (NIPS). – 2016. – Vol. 29. – P. 3315–3323.
28. Kleinberg, J. Inherent Trade-Offs in the Fair Determination of Risk / J. Kleinberg, S. Mullainathan, M. Raghavan // Proceedings of Innovations in Theoretical Computer Science (ITCS). – 2017. – P. 43–67.
29. Lundberg, S. M. A Unified Approach to Interpreting Model Predictions / S. M. Lundberg, S. I. Lee // Advances in Neural Information Processing Systems. – 2017. – P. 4765–4774.
30. Madry, A. Towards Deep Learning Models Resistant to Adversarial Attacks / A. Madry, A. Makelov, L. Schmidt [et al.] // International Conference on Learning Representations (ICLR). – 2018. – P. 1–28.
31. Mehrabi, N. A Survey on Bias and Fairness in Machine Learning / N. Mehrabi, F. Morstatter, N. Saxena [et al.] // ACM Computing Surveys. – 2021. – Vol. 54, no. 6. – P. 1–35.
32. Ribeiro, M. T. "Why Should I Trust You?": Explaining the Predictions of Any Classifier / M. T. Ribeiro, S. Singh, C. Guestrin // Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. – 2016. – P. 1135–1144.
33. Verma, S. Fairness Definitions Explained / S. Verma, J. Rubin // Proceedings of the International Workshop on Software Fairness (FairWare). – 2018. –

P. 1–7.

34. Wadsworth, C. Achieving Fairness through Adversarial Learning: an Application to Recidivism Prediction / C. Wadsworth, F. Vera, C. Piech // arXiv preprint arXiv:1807.00199. – 2018.
35. Zafar, M. B. Fairness Constraints: Mechanisms for Fair Classification / M. B. Zafar, I. Valera, M. Gomez Rodriguez, K. P. Gummadi // Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS). – 2017. – P. 962–970.
36. Zhang, B. H. Mitigating Unwanted Biases with Adversarial Learning / B. H. Zhang, B. Lemoine, M. Mitchell // Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society. – 2018. – P. 335–340.

IV. Электронные ресурсы и техническая документация

37. Celery: Distributed Task Queue [Электронный ресурс] : официальная документация. – URL: <https://docs.celeryq.dev/en/stable/> (дата обращения: 20.02.2025).
38. FastAPI: Modern (fast) web framework for building APIs with Python [Электронный ресурс]. – URL: <https://fastapi.tiangolo.com/> (дата обращения: 20.02.2025).
39. MITRE ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems) [Электронный ресурс]. – URL: <https://atlas.mitre.org/> (дата обращения: 10.02.2025).
40. NIST AI Risk Management Framework (AI RMF 1.0) [Электронный ресурс]. – URL: <https://www.nist.gov/itl/ai-risk-management-framework> (дата обращения: 15.01.2025).
41. OWASP Machine Learning Security Top 10 [Электронный ресурс]. – URL: <https://owasp.org/www-project-machine-learning-security-top-10/> (дата обращения: 01.02.2025).
42. PyTorch Documentation [Электронный ресурс]. – URL: <https://pytorch.org/docs/stable/index.html> (дата обращения: 20.02.2025).
43. Redis Documentation [Электронный ресурс]. – URL: <https://redis.io/docs/> (дата обращения: 18.02.2025).
44. The UCI Machine Learning Repository: Adult Data Set [Электронный ресурс]. – URL: <https://archive.ics.uci.edu/dataset/2/adult> (дата обращения: 10.02.2025).

45. Uvicorn: An ASGI web server implementation for Python [Электронный ресурс]. – URL: <https://www.uvicorn.org/> (дата обращения: 22.02.2025).

ПРИЛОЖЕНИЕ А

Модуль состязательного обучения (fair_train_mine.py)

```
1 import os
2 import random
3 import logging
4 import numpy as np
5 import torch
6 import torch.nn as nn
7 import torch.optim as optim
8 from torch.utils.data import TensorDataset, DataLoader
9 from sklearn.metrics import accuracy_score, f1_score
10
11 # Logger setup
12 logging.basicConfig(level=logging.INFO, format='%(asctime)s |
13     MINE-CORE | %(levelname)s: %(message)s')
14 logger = logging.getLogger(__name__)
15
16 def seed_everything(seed=42):
17     """
18     Ensures experiment determinism.
19     Fixes RNG for weights reproducibility and batching.
20     """
21     random.seed(seed)
22     os.environ['PYTHONHASHSEED'] = str(seed)
23     np.random.seed(seed)
24     torch.manual_seed(seed)
25     if torch.cuda.is_available():
26         torch.cuda.manual_seed(seed)
27         torch.cuda.manual_seed_all(seed)
28         torch.backends.cudnn.deterministic = True
29         torch.backends.cudnn.benchmark = False
30     logger.info(f"Random Seed fixed: {seed}")
31
32 class Predictor(nn.Module):
33     """
34     Main classifier network.
```

```

34     nn.Sigmoid() removed for numerical stability (LogSumExp
        trick).
35     """
36     def __init__(self, input_dim: int):
37         super(Predictor, self).__init__()
38         self.net = nn.Sequential(
39             nn.Linear(input_dim, 64),
40             nn.ReLU(),
41             nn.Dropout(0.2), # Dropout added to prevent
                overfitting
42             nn.Linear(64, 32),
43             nn.ReLU(),
44             nn.Linear(32, 1)
45         )
46
47     def forward(self, x):
48         # Returns unnormalized logits
49         return self.net(x)
50
51 class MultiMINE(nn.Module):
52     """
53     Mutual Information Estimator (Critic).
54     Estimates dependence I(Y_pred; Z).
55     """
56     def __init__(self, z_dim: int):
57         super(MultiMINE, self).__init__()
58         # Input: prediction probability (1) + protected
                attribute (z_dim)
59         self.net = nn.Sequential(
60             nn.Linear(1 + z_dim, 64),
61             nn.ReLU(),
62             nn.Linear(64, 32),
63             nn.ReLU(),
64             nn.Linear(32, 1)
65         )
66
67     def forward(self, y_pred, z):
68         return self.net(torch.cat([y_pred, z], dim=1))
69
70 class MINEAdversarialTrainer:
71     """

```

```

72 Adversarial Training Orchestrator.
73 Implements minimax game.
74 """
75 def __init__(self, input_dim: int, z_dim: int = 1,
76               lambda_reg: float = 15.0, lr: float = 1e-3, seed: int =
77               42):
78     seed_everything(seed)
79
80     self.device = torch.device("cuda" if
81                               torch.cuda.is_available() else "cpu")
82     self.lambda_reg = lambda_reg
83
84     self.predictor = Predictor(input_dim).to(self.device)
85     self.critic = MultiMINE(z_dim).to(self.device)
86
87     self.opt_p = optim.Adam(self.predictor.parameters(),
88                             lr=lr)
89     self.opt_c = optim.Adam(self.critic.parameters(),
90                             lr=lr)
91
92     # Scheduler decreases LR if loss plateaus
93     self.scheduler = optim.lr_scheduler.ReduceLROnPlateau(
94         self.opt_p, mode='min', factor=0.5, patience=5
95     )
96     logger.info(f"Trainer init. Device: {self.device}.
97                 Lambda: {lambda_reg}")
98
99 def _mine_loss(self, y_pred_prob, z):
100     """
101     Calculates Donsker-Varadhan lower bound.
102     """
103     t_joint = self.critic(y_pred_prob, z)
104
105     # Shuffle z for marginal distribution
106     idx = torch.randperm(z.size(0), device=self.device)
107     z_shuffled = z[idx]
108     t_marginal = self.critic(y_pred_prob, z_shuffled)
109
110     # Add 1e-6 for numerical stability of logarithm
111     mi = torch.mean(t_joint) -
112         torch.log(torch.mean(torch.exp(t_marginal)) + 1e-6)

```

```

106         return mi
107
108     def fit(self, X: np.ndarray, y: np.ndarray, z: np.ndarray,
109           epochs: int = 50, batch_size: int = 1024):
110         """Main training loop."""
111         X_t = torch.FloatTensor(X)
112         y_t = torch.FloatTensor(y).reshape(-1, 1)
113         z_t = torch.FloatTensor(z).reshape(-1, 1)
114
115         # Class imbalance handling
116         num_pos = np.sum(y == 1)
117         num_neg = np.sum(y == 0)
118         pos_weight_val = num_neg / max(num_pos, 1)
119
120         pos_weight_tensor =
121             torch.FloatTensor([pos_weight_val]).to(self.device)
122         logger.info(f"Class Imbalance Correction: pos_weight =
123             {pos_weight_val:.4f}")
124         criterion_task =
125             nn.BCEWithLogitsLoss(pos_weight=pos_weight_tensor)
126
127         dataset = TensorDataset(X_t, y_t, z_t)
128         dataloader = DataLoader(dataset,
129             batch_size=batch_size, shuffle=True)
130         history = {"task_loss": [], "mi_loss": [], "f1_score":
131             []}
132
133         self.predictor.train()
134         self.critic.train()
135
136         for epoch in range(epochs):
137             epoch_task_loss = 0.0
138             epoch_mi = 0.0
139             all_preds = []
140             all_targets = []
141
142             for batch_idx, (bx, by, bz) in
143                 enumerate(dataloader):
144                 bx, by, bz = bx.to(self.device),
145                     by.to(self.device), bz.to(self.device)

```



```

139         # Step 1: Critic Training (k=5)
140         for _ in range(5):
141             self.opt_c.zero_grad()
142             with torch.no_grad():
143                 logits = self.predictor(bx)
144                 probs = torch.sigmoid(logits)
145
146                 mi = self._mine_loss(probs, bz)
147                 loss_c = -mi
148                 loss_c.backward()
149                 self.opt_c.step()
150
151         # Step 2: Predictor Training (k=1)
152         self.opt_p.zero_grad()
153         logits = self.predictor(bx)
154         probs = torch.sigmoid(logits)
155
156         task_loss = criterion_task(logits, by)
157         mi_val = self._mine_loss(probs, bz)
158
159         total_loss = task_loss + (self.lambda_reg *
160                                 mi_val)
161         total_loss.backward()
162         self.opt_p.step()
163
164         epoch_task_loss += task_loss.item()
165         epoch_mi += mi_val.item()
166
167         preds_bin = (probs > 0.5).long()
168         all_preds.append(preds_bin.cpu().numpy())
169         all_targets.append(by.cpu().numpy())
170
171         # Epoch wrap-up
172         avg_task_loss = epoch_task_loss / len(dataloader)
173         avg_mi = epoch_mi / len(dataloader)
174
175         all_preds = np.concatenate(all_preds)
176         all_targets = np.concatenate(all_targets)
177
178         epoch_acc = accuracy_score(all_targets, all_preds)
179         epoch_f1 = f1_score(all_targets, all_preds)

```

```

179
180         history["task_loss"].append(avg_task_loss)
181         history["mi_loss"].append(avg_mi)
182         history["f1_score"].append(epoch_f1)
183
184         self.scheduler.step(avg_task_loss)
185
186         if (epoch + 1) % 1 == 0:
187             logger.info(f"Epoch {epoch+1:02d} | Loss:
188                         {avg_task_loss:.4f} | MI: {avg_mi:.4f} |
189                         F1: {epoch_f1:.4f}")
190
191         return history
192
193     def save_model(self, path: str):
194         torch.save(self.predictor.state_dict(), path)
195         logger.info(f"Model saved: {path}")
196
197 if __name__ == "__main__":
198     X_dummy = np.random.randn(100, 10).astype(np.float32)
199     y_dummy = np.random.randint(0, 2,
200                                size=(100,)).astype(np.float32)
201     z_dummy = np.random.randint(0, 2,
202                                size=(100,)).astype(np.float32)
203     trainer = MINEAdversarialTrainer(input_dim=10)
204     trainer.fit(X_dummy, y_dummy, z_dummy, epochs=2,
205                batch_size=10)

```

ПРИЛОЖЕНИЕ Б

Модуль обеспечения безопасности (security_manager.py)

```
1 import hashlib
2 import os
3 import logging
4 import pickle
5 import torch
6
7 # Logger setup
8 logging.basicConfig(level=logging.INFO, format='%(asctime)s |
    SECURITY | %(levelname)s: %(message)s')
9 logger = logging.getLogger(__name__)
10
11 class SecurityManager:
12     """
13     ML artifacts integrity control module.
14     Verifies SHA-256 hashes of models.
15     """
16     @staticmethod
17     def get_hash(file_path: str) -> str:
18         """Calculates SHA-256 hash."""
19         sha = hashlib.sha256()
20         try:
21             with open(file_path, "rb") as f:
22                 # Read in 4KB chunks
23                 while chunk := f.read(4096):
24                     sha.update(chunk)
25             return sha.hexdigest()
26         except FileNotFoundError:
27             logger.error(f"File not found: {file_path}")
28             return ""
29
30     @staticmethod
31     def verify(file_path: str, expected: str = None) -> str:
32         """Verifies file integrity against expected hash."""
33         current = SecurityManager.get_hash(file_path)
```

```

34         if expected:
35             if current != expected:
36                 logger.critical(f"SECURITY ALERT: Expected
37                     {expected[:8]}, got {current[:8]}")
38                 raise ValueError("Integrity Check Failed:
39                     Model file modified!")
40             else:
41                 logger.info(f"Integrity OK. Hash verified.")
42         return current
43
44 class SafeUnpickler(pickle.Unpickler):
45     """
46     Secure deserializer protecting against RCE attacks (Pickle
47     Bomb).
48     Only allows approved modules (Allowlist).
49     """
50     SAFE_MODULES = {
51         'torch', 'collections', 'numpy',
52         'numpy.core.multiarray',
53         'torch.nn', 'torch.utils', 'torch._utils', 're'
54     }
55
56     def find_class(self, module, name):
57         if module.split('.')[0] not in self.SAFE_MODULES:
58             logger.warning(f"SECURITY ALERT: Blocked unsafe
59                 module '{module}.{name}'")
60             raise pickle.UnpicklingError(f"Module
61                 '{module}.{name}' forbidden by Policy.")
62         return super().find_class(module, name)
63
64 def safe_load_model(file_path: str):
65     """Secure wrapper for torch.load."""
66     if not os.path.exists(file_path):
67         raise FileNotFoundError(f"Model file not found:
68             {file_path}")
69
70     SecurityManager.verify(file_path)
71     with open(file_path, 'rb') as f:
72         return torch.load(f, pickle_module=SafeUnpickler)

```

ПРИЛОЖЕНИЕ В

Сервис API-шлюза (main.py)

```
1 import os
2 import uvicorn
3 import logging
4 from fastapi import FastAPI, HTTPException
5 from fastapi.middleware.cors import CORSMiddleware
6 from celery.result import AsyncResult
7 from contextlib import asynccontextmanager
8
9 from schemas import TrainingRequest, AuditRequest, TaskResponse
10 from tasks import train_fair_model_task, run_audit_task,
11     celery_app
12 from security_manager import SecurityManager
13
14 logging.basicConfig(level=logging.INFO, format='%(asctime)s |
15     API | %(levelname)s: %(message)s')
16 logger = logging.getLogger(__name__)
17
18 @asynccontextmanager
19 async def lifespan(app: FastAPI):
20     """Infrastructure initialization."""
21     os.makedirs("models", exist_ok=True)
22     os.makedirs("data", exist_ok=True)
23     logger.info("API running. Directories verified.")
24     yield
25
26 app = FastAPI(
27     title="Trustworthy AI Platform",
28     description="API for MINE and fairness audit.",
29     version="2.0.0",
30     lifespan=lifespan
31 )
32
33 app.add_middleware(
34     CORSMiddleware,
35     allow_origins=["*"],
36     allow_methods=["*"],
```

```

35     allow_headers=["*"],
36 )
37
38 @app.post("/train", response_model=TaskResponse)
39 async def train_model(request: TrainingRequest):
40     """Asynchronous training start."""
41     if not os.path.exists(request.dataset_path):
42         raise HTTPException(status_code=404, detail="File not
43             found")
44
45     task = train_fair_model_task.delay(
46         data_path=request.dataset_path,
47         params=request.model_dump()
48     )
49     return TaskResponse(task_id=task.id, status="PENDING")
50
51 @app.post("/audit", response_model=TaskResponse)
52 async def audit_model(request: AuditRequest):
53     """Asynchronous audit start."""
54     if not os.path.exists(request.model_path):
55         raise HTTPException(status_code=404, detail="Model not
56             found")
57
58     task = run_audit_task.delay(
59         model_path=request.model_path,
60         dataset_path=request.dataset_path,
61         config=request.model_dump()
62     )
63     return TaskResponse(task_id=task.id, status="PENDING")
64
65 @app.get("/status/{task_id}")
66 async def get_task_status(task_id: str):
67     """Get task status."""
68     task_result = AsyncResult(task_id, app=celery_app)
69     response = {"task_id": task_id, "status":
70         task_result.status, "result": None}
71
72     if task_result.ready():
73         if task_result.successful():
74             response["result"] = task_result.result
75         else:

```

```

73         response["status"] = "FAILURE"
74         response["error"] = str(task_result.info)
75     return response
76
77 @app.get("/security/verify/{model_filename}")
78 async def verify_model_hash(model_filename: str):
79     """Direct integrity check call."""
80     file_path = os.path.join("models", model_filename)
81     if not os.path.exists(file_path):
82         raise HTTPException(status_code=404, detail="File not
            found")
83     computed_hash = SecurityManager.get_hash(file_path)
84     return {"filename": model_filename, "sha256":
            computed_hash, "message": "Integrity passed"}
85
86 if __name__ == "__main__":
87     uvicorn.run(app, host="127.0.0.1", port=8000)

```

ПРИЛОЖЕНИЕ Г

Модуль асинхронной обработки (tasks.py)

```
1 import os
2 import time
3 import logging
4 import torch
5 import pandas as pd
6 from typing import Dict, Any
7 from celery import Celery
8
9 from fair_train_mine import MINEAdversarialTrainer, Predictor
10 from auditor import FairnessAuditor
11 from security_manager import SecurityManager
12
13 logger = logging.getLogger(__name__)
14
15 celery_app = Celery("tasks",
16                     broker="redis://localhost:6379/0",
17                     backend="redis://localhost:6379/0")
18 celery_app.conf.update(
19     task_serializer='json',
20     accept_content=['json'],
21     result_serializer='json',
22     worker_pool='solo',
23     task_track_started=True
24 )
25
26 @celery_app.task(bind=True, name="tasks.train_fair_model")
27 def train_fair_model_task(self, data_path: str, params:
28     Dict[str, Any]):
29     """MINE model training task."""
30     try:
31         if not os.path.exists(data_path):
32             raise FileNotFoundError("Dataset not found.")
33
34         df = pd.read_csv(data_path)
35         features = params['features']
36         target = params['target']
```



```

34     protected = params['protected']
35
36     X = df[features].values
37     y = df[target].values
38     z = df[protected].values
39
40     trainer = MINEAdversarialTrainer(
41         input_dim=len(features),
42         z_dim=1,
43         lambda_reg=params.get("lambda_reg", 15.0)
44     )
45
46     history = trainer.fit(X, y, z,
47                          epochs=params.get("epochs", 20))
48
49     os.makedirs("models", exist_ok=True)
50     model_path = os.path.join("models",
51                               f"fair_model_{int(time.time())}.pth")
52     trainer.save_model(model_path)
53
54     model_hash = SecurityManager.get_hash(model_path)
55
56     return {
57         "status": "COMPLETED",
58         "model_path": model_path,
59         "model_hash": model_hash
60     }
61 except Exception as exc:
62     logger.error(f"Training error: {exc}")
63     raise exc
64
65 @celery_app.task(name="tasks.run_audit")
66 def run_audit_task(model_path: str, dataset_path: str, config:
67                    Dict[str, Any]):
68     """Fairness audit task."""
69     try:
70         expected_hash = config.get("expected_hash")
71         if expected_hash:
72             SecurityManager.verify(model_path, expected_hash)

```

```

71     state_dict = torch.load(model_path,
72                               map_location='cpu', weights_only=True)
73     model = Predictor(len(config['features']))
74     model.load_state_dict(state_dict)
75     model.eval()
76
77     df = pd.read_csv(dataset_path)
78     X_tensor =
79         torch.FloatTensor(df[config['features']].values)
80
81     with torch.no_grad():
82         preds = (torch.sigmoid(model(X_tensor)) >
83                 0.5).int().numpy().flatten()
84
85     audit_df = pd.DataFrame({
86         'prediction': preds,
87         config['protected']: df[config['protected']].values
88     })
89
90     y_true_col = None
91     if config['target'] in df.columns:
92         audit_df['ground_truth'] =
93             df[config['target']].values
94         y_true_col = 'ground_truth'
95
96     auditor = FairnessAuditor(audit_df, 'prediction',
97                               config['protected'])
98     metrics =
99         auditor.get_full_report(y_true_col=y_true_col)
100
101     return {
102         "status": "AUDIT_COMPLETED",
103         "metrics": metrics,
104         "verified": True if expected_hash else False
105     }
106 except Exception as exc:
107     return {"status": "FAILED", "error": str(exc)}

```

ПРИЛОЖЕНИЕ Д

Модуль расчета метрик (auditor.py)

```
1 import pandas as pd
2 import numpy as np
3 import logging
4 from typing import Dict
5 from sklearn.metrics import accuracy_score, f1_score,
6     confusion_matrix
7
8
9 logger = logging.getLogger(__name__)
10
11 class FairnessAuditor:
12     """Professional fairness audit module."""
13     def __init__(self, dataframe: pd.DataFrame, target_col:
14         str, protected_col: str):
15         self.df = dataframe.copy()
16         self.y = target_col
17         self.z = protected_col
18
19         if self.y not in self.df.columns or self.z not in
20             self.df.columns:
21             raise ValueError("Required columns not found.")
22
23     def get_full_report(self, y_true_col: str = None) ->
24         Dict[str, float]:
25         """Generates full fairness and accuracy report using
26             vectorized operations."""
27         group_p = self.df[self.df[self.z] == 1]
28         group_u = self.df[self.df[self.z] == 0]
29
30         if len(group_p) == 0 or len(group_u) == 0:
31             logger.warning("Empty group found. Cannot
32                 calculate metrics.")
33             return {}
34
35         sr_p = group_p[self.y].mean()
36         sr_u = group_u[self.y].mean()
```

```

31     spd = sr_u - sr_p
32     di = sr_u / sr_p if sr_p > 0 else 0.0
33
34     metrics = {
35         "accuracy": 0.0,
36         "f1_score": 0.0,
37         "statistical_parity_difference": round(float(spd),
38             4),
39         "disparate_impact": round(float(di), 4),
40         "selection_rate_privileged": round(float(sr_p), 4),
41         "selection_rate_unprivileged": round(float(sr_u),
42             4),
43         "equal_opportunity_difference": 0.0
44     }
45
46     if y_true_col and y_true_col in self.df.columns:
47         y_true = self.df[y_true_col]
48         y_pred = self.df[self.y]
49
50         metrics["accuracy"] = round(accuracy_score(y_true,
51             y_pred), 4)
52         metrics["f1_score"] = round(f1_score(y_true,
53             y_pred), 4)
54
55         tpr_p = self._get_tpr(group_p, y_true_col)
56         tpr_u = self._get_tpr(group_u, y_true_col)
57         metrics["equal_opportunity_difference"] =
58             round(tpr_u - tpr_p, 4)
59
60     return metrics
61
62 def _get_tpr(self, group_df: pd.DataFrame, y_true_col:
63     str) -> float:
64     """Helper method for TPR calculation."""
65     y_true = group_df[y_true_col]
66     y_pred = group_df[self.y]
67
68     if sum(y_true) == 0:
69         return 0.0

```

```
65         tn, fp, fn, tp = confusion_matrix(y_true, y_pred,  
66             labels=[0, 1]).ravel()  
        return tp / (tp + fn)
```

ПРИЛОЖЕНИЕ Е

Контракты данных API (schemas.py)

```
1 from pydantic import BaseModel, Field, field_validator
2 from typing import List, Optional, Dict
3 from datetime import datetime
4
5 class TrainingRequest(BaseModel):
6     """Request for adversarial training."""
7     dataset_path: str = Field(..., description="Path to CSV
8         dataset")
9     features: List[str] = Field(..., min_length=1)
10    target: str = Field(..., description="Target variable")
11    protected: str = Field(..., description="Sensitive
12        attribute")
13    lambda_reg: float = Field(default=15.0, ge=0.0, le=50.0,
14        description="MINE coefficient")
15    epochs: int = Field(default=20, ge=1, le=1000)
16
17    @field_validator('dataset_path')
18    @classmethod
19    def validate_csv(cls, v: str) -> str:
20        if not v.lower().endswith('.csv'):
21            raise ValueError("File must be .csv")
22        return v
23
24 class FairnessMetrics(BaseModel):
25     accuracy: float
26     f1_score: float
27     statistical_parity_difference: float
28     disparate_impact: float
29     equal_opportunity_difference: float
30     selection_rate_privileged: float
31     selection_rate_unprivileged: float
32
33 class AuditRequest(BaseModel):
34     """Request for technical audit."""
35     model_path: str = Field(...)
36     dataset_path: str = Field(...)
```

```
34     features: List[str]
35     target: str
36     protected: str
37     expected_hash: Optional[str] = Field(None,
38         description="SHA-256 integrity hash")
39
40 class TaskResponse(BaseModel):
41     task_id: str = Field(...)
42     status: str = Field(default="PENDING")
43     created_at: datetime = Field(default_factory=datetime.now)
44
45 class AuditResult(BaseModel):
46     status: str
47     metrics: FairnessMetrics
48     model_hash: str
49     timestamp: float = Field(default_factory=lambda:
50         datetime.now().timestamp())
```

ПРИЛОЖЕНИЕ Ж

Скрипт автоматизации эксперимента (run_tasks.py)

```
1 import logging
2 from tasks import train_fair_model_task, run_audit_task
3
4 logging.basicConfig(level=logging.INFO, format='%(asctime)s |
   EXPERIMENT | %(message)s')
5 logger = logging.getLogger(__name__)
6
7 def run_experiment_pipeline():
8     """Automated A/B testing scenario."""
9     logger.info("--- EXPERIMENT PIPELINE STARTED ---")
10
11     DATA_PATH = "data/adult_processed.csv"
12     TEST_DATA_PATH = "data/adult_processed.csv"
13
14     config = {
15         "features": ['age', 'education-num', 'capital-gain',
16                     'capital-loss', 'hours-per-week'],
17         "target": "target",
18         "protected": "sex_binary",
19         "lambda_reg": 15.0,
20         "epochs": 30,
21         "batch_size": 1024
22     }
23
24     logger.info(f"Starting training with
25                 lambda={config['lambda_reg']}.")
26     try:
27         train_result =
28             train_fair_model_task.apply(args=[DATA_PATH,
29                                               config]).get()
30     except Exception as e:
31         logger.error(f"Training error: {e}")
32     return
```



```

30     if train_result["status"] != "COMPLETED":
31         logger.error("Training failed.")
32         return
33
34     model_path = train_result["model_path"]
35     model_hash = train_result["model_hash"]
36     logger.info(f"Model saved: {model_path} | Hash:
37         {model_hash}")
38
39     logger.info("Starting audit...")
40     audit_config = {
41         "features": config["features"],
42         "target": config["target"],
43         "protected": config["protected"],
44         "expected_hash": model_hash
45     }
46
47     try:
48         audit_result = run_audit_task.apply(args=[model_path,
49             TEST_DATA_PATH, audit_config]).get()
50     except Exception as e:
51         logger.error(f"Audit error: {e}")
52         return
53
54     metrics = audit_result["metrics"]
55     logger.info("-" * 30)
56     logger.info("FINAL REPORT:")
57     logger.info(f"Accuracy: {metrics.get('accuracy',
58         0.0):.4f}")
59     logger.info(f"F1-Score: {metrics.get('f1_score',
60         0.0):.4f}")
61     logger.info(f"Disparate Impact:
62         {metrics.get('disparate_impact', 0.0):.4f}")
63     logger.info(f"SPD:
64         {metrics.get('statistical_parity_difference',
65         0.0):.4f}")
66     logger.info("-" * 30)
67
68     if metrics['disparate_impact'] >= 0.8:
69         logger.info("SUCCESS: Model complies with fairness
70             standards (DI >= 0.8).")

```

```
63     else:
64         logger.warning("FAILURE: Discrimination detected.")
65
66 if __name__ == "__main__":
67     run_experiment_pipeline()
```

ПРИЛОЖЕНИЕ 3

Скрипт имитации атак и тестирования контура безопасности (test_security.py)

```
1 import os
2 import pickle
3 import torch
4 from security_manager import SecurityManager, SafeUnpickler
5
6 # -----
7 # SCENARIO 1: RCE Attack Simulation (Pickle Bomb)
8 # -----
9 class MaliciousPayload(object):
10     """Malicious class attempting to execute system command"""
11     def __reduce__(self):
12         return (os.system, ('echo "HACKED! YOUR SYSTEM IS
13             COMPROMISED!"',))
14
15 print("\n=== TEST 1: RCE PROTECTION (PICKLE BOMB) ===")
16 malicious_file = "models/hacked_model.pth"
17
18 # 1. Create "infected" model file
19 os.makedirs("models", exist_ok=True)
20 with open(malicious_file, "wb") as f:
21     pickle.dump(MaliciousPayload(), f)
22
23 print(f"[!] Malicious file created: {malicious_file}")
24 print("[*] Attempting secure deserialization
25     (SafeUnpickler)...")
26
27 # 2. Try to load it directly via our secure parser
28 try:
29     with open(malicious_file, 'rb') as f:
30         SafeUnpickler(f).load()
31 except pickle.UnpicklingError as e:
32     print(f"[+] ATTACK BLOCKED! Error: {e}")
33
34 # -----
```

```

33 # SCENARIO 2: Model Tampering Simulation
34 # -----
35 print("\n=== TEST 2: INTEGRITY CONTROL (SHA-256) ===")
36 fake_model_file = "models/fake_weights.pth"
37
38 # 1. Create fake model
39 with open(fake_model_file, "wb") as f:
40     f.write(b"fake_tensor_weights_123")
41
42 # 2. Attacker knows the system expects this reference hash
43 expected_hash = (
44     "d38f34a4c6b92d3b58388babd74b4e79 "
45     "ee18d237765ff568aabde6439a42393c "
46 )
47
48 print(f"[*] Attempting to load modified file with reference\n    hash...")
49 try:
50     SecurityManager.verify(fake_model_file,
51                             expected=expected_hash)
52 except ValueError as e:
53     print(f"[+] ATTACK BLOCKED! Error: {e}")

```